



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Dipartimento di Informatica - Scienza e Ingegneria - DISI

Tesi di Laurea Triennale

Deep Reinforcement Learning e volatility
targeting nella gestione dinamica
dell'esposizione: un'analisi dei limiti
dell'apprendimento

Relatore:
Prof. Federico Montori

Candidato:
Cesidio Rosica

Mese Sessione di Laurea Ottobre
Anno Accademico 2025/2026

Riassunto

Questo lavoro indaga se la componente appresa di un agente di Deep Reinforcement Learning aggiunga valore rispetto a una regola strutturale fissa che punta a regolare l'esposizione al mercato. L'agente addestrato con PPO su una rete ricorrente (GRU) e con politica basata su una distribuzione Beta, non sostituisce la strategia di volatility targeting di Moreira e Muir ma la corregge attraverso un unico moltiplicatore appreso m_θ . La selezione dei titoli è congelata a equal-weight in modo da isolare la sola decisione di esposizione e permettere un confronto pulito tra ciò che la rete impara e ciò che la formula fornisce.

Il modello è valutato out-of-sample su più regimi di mercato e su panieri diversi, con protocollo multi-seed e una frontiera di controllo a parità di liquidità media. I risultati mostrano che la parte appresa non aggiunge nulla che non si possa ottenere scegliendo i parametri strutturali fissi. Il minor drawdown dell'agente deriva dal mantenere più liquidità, non da un reale tempismo, l'agente quindi non ha sviluppato alcun tipo d'intelligenza di market timing. L'agente ricade sulla frontiera della regola deterministica, resta dominato sullo Sharpe e la dipendenza dal regime dell'esponente non è sfruttabile in tempo reale. La ragione è economica: la volatilità è prevedibile, la direzione del mercato no.

Il contributo non è dimostrare che l'apprendimento batte una regola, ma quantificare con rigore, tramite ablation e controlli, il limite dell'adattività appresa in un dominio a basso rapporto segnale/rumore. Come obiettivo secondario è stata sviluppata un'applicazione mobile local-first che esegue il modello on-device, rendendo le strategie studiate concretamente utilizzabili.

Contents

Riassunto	i
1 Introduzione	1.0
1.1 Stato dell'arte	1.1
1.2 Definizione del problema	1.1
1.2.1 Obiettivi e limiti	1.4
2 Fondamenti Teorici	2.0
2.1 Fondamenti economici	2.0
2.1.1 Gestione del portafoglio	2.0
2.1.2 Volatilità e drawdown	2.0
2.1.3 Volatility targeting	2.1
2.1.4 Dividendi	2.1
2.1.5 Frazionamento azionario (Stock Split)	2.1
2.1.6 Regimi di mercato	2.2
2.2 Fondamenti Deep Reinforcement Learning	2.2
2.2.1 Reinforcement learning	2.2
2.2.2 Metodi Actor-Critic e PPO	2.3
2.2.3 Distribuzione beta	2.3
2.2.4 Reti ricorrenti e invarianza di dominio	2.4
2.2.5 Apprendimento supervisionato	2.4
2.2.6 Binary Cross-Entropy Loss (BCE)	2.5
3 Progettazione dataset e metodologia sperimentale	3.0
3.1 Progettazione del dataset	3.0
3.1.1 Fonte e universo	3.0
3.1.2 Feature e loro selezione	3.1
3.1.3 Suddivisione dei dati	3.1
3.2 Metodologia sperimentale	3.1
3.2.1 Strategia appresa e formula di riferimento	3.1
3.2.2 Funzione di reward	3.2
3.2.3 Protocollo di valutazione	3.2

3.2.4	Baseline di confronto	3.3
3.2.5	Metriche di valutazione	3.3
4	Implementazione modello	4.0
4.1	Modello	4.0
4.1.1	Introduzione	4.0
4.1.2	Feature	4.0
4.1.3	Architettura	4.2
4.1.4	Environment	4.5
4.1.5	Addestramento	4.7
5	Esperimenti e Risultati	5.0
5.1	Conclusioni: cosa aggiunge l'apprendimento	5.0
5.1.1	Esperimento 1: Come influisce il parametro σ_{target}	5.0
5.1.2	Esperimento 2: L'esponente al centro delle osservazioni	5.3
5.1.3	Generalità: trasferimento su panieri diversi	5.8
5.1.4	Filo conduttore	5.10
6	Applicazione Mobile	6.0
6.1	Obiettivo dell'applicazione	6.0
6.2	Interfaccia e funzionalità	6.1
6.2.1	Dashboard e gestione del portafoglio	6.1
6.2.2	Elenco dei titoli	6.4
6.2.3	Dettaglio del titolo	6.5
6.3	Dettagli implementativi	6.5
6.3.1	Datasets	6.6
6.3.2	Esecuzione del modello on-device	6.8
6.3.3	Motore di backtest	6.9
6.3.4	Validazione numerica	6.10
6.4	Strategie	6.10
6.4.1	Equipesato	6.11
6.4.2	Inverse Volatility	6.11
6.4.3	Kelly Criterion	6.11
6.4.4	Volatility Targeting	6.13
7	Conclusioni	7.0
7.1	Discussione: un risultato atteso nella letteratura	7.0
	Bibliography	B1
	Acknowledgment	B3

Chapter 1

Introduzione

Il position sizing, ovvero la decisione di quanto capitale investire in un dato titolo e momento, costituisce una variabile importantissima per la preservazione del capitale e la stabilità dei rendimenti nel lungo periodo. Un guadagno molto grande derivato da un titolo su cui abbiamo investito una quantità piccola del nostro capitale risulterà essere marginale nel valore complessivo del portafoglio. Mentre allocare gran parte del capitale su un titolo che è molto instabile potrebbe compromettere l'intero portafoglio. Per questo motivo è molto importante spartire il proprio capitale in maniera sensata nel momento in cui si va a investire. Nel lavoro di Blotnick [3] si affrontano alcuni problemi che possono incorrere quando si prova a fare position sizing affidandosi al puro intuito, come:

- **Eccesso di fiducia:** porta a concentrare eccessivamente il capitale sui titoli che si è convinti vadano bene, sottostimando le probabilità di crollo.
- **Avversione alle perdite:** è la tendenza a vendere le proprie posizioni in profitto per incassare il prima possibile il guadagno, lasciando d'altra parte ferme le posizioni in perdita sperando in un recupero aumentandone così il peso complessivo nel portafoglio.
- **Pregiudizio di recency:** è ciò che induce a investire di più in fasi rialziste prolungate e a disinvestire drasticamente nei ribassi successivi.

Per evitare questi problemi, la ricerca finanziaria ha sviluppato metodologie di dimensionamento della posizione. I primi approcci sono stati quelli a capitale fisso come l'Equal Weight in cui si divide semplicemente il capitale equamente fra i titoli scelti. Questi metodi sono molto semplici e addirittura profittevoli ma non offrono alcuna protezione dai rischi di crollo del mercato. Il mercato azionario infatti è contrassegnato da repentini cambiamenti di stato e dalla tendenza della volatilità a persistere nel tempo. Osservando questa struttura è diventato necessario sviluppare metodologie di posizionamento dinamiche guidate dal rischio. Questo lavoro si è basato sulla formula del volatility targeting proposta da Moreira e Muir [13]. Essa scala l'esposizione al mercato in proporzione inversa alla

volatilità stimata cioè investe nei periodi in cui la volatilità è bassa e ritorna in liquidità nei momenti in cui quest'ultima si alza.

1.1 Stato dell'arte

Durante gli anni la gestione del portafoglio ha attraversato una profonda evoluzione. Si è partiti da modelli analitici statici fino ad arrivare ad architetture adattive basate sull'intelligenza artificiale. Tradizionalmente l'allocazione del capitale si è fondata su modelli di ottimizzazione media-varianza di Markowitz, essi consistevano nel capire quanta percentuale di capitale investire in ogni titolo in base alla media dei rendimenti attesi e alla loro volatilità, cercando l'allocazione ottimale che massimizzi la prima e minimizzi la seconda. In seguito si è evoluta verso politiche focalizzate sulla gestione del rischio come il Volatility Targeting. Queste formule hanno riscontrato un grande successo in quanto sono trasparenti è facile capire il perché delle loro azioni, e si sono dimostrate molto efficienti nell'obiettivo di proteggere il capitale. Tuttavia esse si basano su regole fisse che non riescono ad adattarsi dinamicamente ai velocissimi cambi di regime del mercato. Per far fronte a questo problema negli ultimi dieci anni la ricerca si è mossa verso l'applicazione del Deep Reinforcement Learning (DRL). Questa a differenza dell'apprendimento supervisionato non tenta di prevedere le direzioni future del mercato che si è dimostrato inefficace, ma fa sì che un'agente agisca direttamente con l'ambiente di mercato apprendendo una politica di allocazione tramite tentativi ed errori per massimizzare un segnale di reward, quest'ultimo tipicamente basato su metriche corrette per il rischio come lo Sharpe o il Calmar. Attualmente lo standard impiega algoritmi Actor-Critic come il Proximal Policy Optimization (PPO), per gestire lo spazio di azioni continue richiesto per calibrare i pesi di un portafoglio. I modelli come DeepTrader[16] e MARS [4] rappresentano la frontiera di questo approccio integrando l'estrazione di feature macroeconomiche e indicatori di rischio direttamente all'interno dello stato osservato dall'agente, questo nel tentativo di rendere l'agente consapevole delle turbolenze presenti in ogni momento nel mercato. Purtroppo però l'attuale stato dell'arte è dominato da architetture monolitiche end-to-end, ovvero architetture enormi che prendono i dati grezzi e forniscono direttamente in output il vettore di allocazione dei pesi. In questo modo però non è possibile identificare con precisione la reale aggiunta dell'apprendimento rendendo impossibile quantificare e giustificare un miglioramento dato da un modello rispetto alle normali formule fisse.

1.2 Definizione del problema

Nonostante al giorno d'oggi vengano pubblicati innumerevoli studi su architetture di Deep Reinforcement Learning per l'allocazione dinamica del portafoglio. La letteratura della finanza quantitativa soffre di una mancanza di rigore metodologico nella fase di

validazione e test. Molti di questi studi infatti tendono a riportare solo dei singoli scenari di backtest in cui l'agente batte il benchmark fissato. Questa impostazione però presenta dei punti critici che sono invece stati toccati in questo lavoro:

- **Scelta dei benchmark:** Molto spesso il confronto avviene solo con strategie passive come l'equal-weight o il buy&hold, e quasi mai a parità di rischio. Per avere dei risultati legittimi il benchmark dovrebbe essere ovviamente coerente con l'obiettivo del modello ma è importante anche che sia confrontato a parità di rischio. Questo confronto equo permette di capire meglio se le performance dei vari modelli siano gonfiate o meno dal fatto che assumano un rischio maggiore nelle loro allocazioni.
- **Varianza e seed multipli:** I risultati sono spesso presentati con un unico backtest vincente senza esplicitare che siano stati fatti test con più seed. Quando si vanno a fare molti backtest provando diverse combinazioni di parametri e paniere è possibile che per puro caso si sia incontrata una configurazione vincente, ma che poi essa non lo sia altrettanto testandola fuori campione. Senza avere dati da più backtest e seed che ci indicano quanto sia stabile il modello non possiamo dire se sia stata fatta una scelta del migliore o meno.
- **Ablation dello spazio di azione:** L'agente è quasi sempre una rete monolitica end-to-end e manca un test che isoli il contributo della componente appresa, ovvero che la confronti con una regola semplice, non appresa, capace di svolgere la stessa funzione. Senza questo controllo non è possibile stabilire se il vantaggio derivi davvero dall'apprendimento o da un effetto banale.

Per dimostrare la veridicità di questa affermazione sono stati presi 5 paper che si avvicinano al modello implementato in questo lavoro. Questi documenti sono stati analizzati per vedere se le criticità riportate di sopra sono presenti almeno in parte in essi. Di sotto si riporta una tabella che sintetizza i risultati di questa ricerca:

Paper / Autori	Benchmark a parità di rischio?	Multi-seed?	Ablation?
Zhang, Zohren & Roberts (2020)	Sì (vol scaling 10%)	Non spec.	Parziale
DeepTrader — Wang et al. (2021)	No	Non spec.	Sì (interna)
Jung & Oh (2025)	No	No (seed 42)	Sì (interna)
Lwele et al. (2025)	No	Non spec.	No
MARS — Chen, Li & Wang (2026)	No	No (seed 42)	Sì (interna)

Da questa ricerca emergono tre osservazioni principali.

1. In nessuno dei cinque lavori è citato un addestramento su più seed indipendenti con medie e intervalli di confidenza; l'unico che fa qualcosa di simile è Jung and Oh [11], con un ricampionamento Moving-Block Bootstrap ex-post.
2. Gli studi di ablation compaiono nei lavori [4, 11, 16], ma restano interni: isolano quale componente, reward o architettura contribuisca, senza però confrontare la componente appresa con una regola semplice, non appresa, capace di svolgere la stessa funzione.
3. Solo Zhang et al. [17] applica un confronto a parità di rischio, normalizzando in modo uniforme la volatilità di tutti i modelli e garantendo così la parità del budget di rischio nel confronto.

Alcuni casi sono particolarmente istruttivi. Lwele et al. [12] mostra come dopo l'addestramento le prestazioni collassano (Sharpe da 1.41 a 0.13), perché l'agente si limita a tagliare la volatilità perdendo rendimento, pura conservatività, per giunta aggravato da un evidente look-ahead bias nella selezione del paniere. Wang et al. [16] e Chen et al. [4] sono i più simili a questo lavoro, hanno una reward orientata al drawdown e un de-risking guidato dallo stato del mercato. La loro ablation però resta interna non isolando il contributo dell'apprendimento rispetto a una regola. Jung and Oh [11] è il più trasparente, ammette che nel mercato delle criptovalute la sovraperformance non sopravvive alla verifica bootstrap.

1.2.1 Obiettivi e limiti

Obiettivo primario

Dopo aver esplorato queste mancanze presenti in molti lavori della finanza quantitativa si è definito lo scopo di questo lavoro. L'obiettivo è esplorare i limiti effettivi dell'intelligenza artificiale nel position sizing, in particolare nel regolare l'esposizione del capitale al mercato. La domanda di questa ricerca non è se una rete riesce a battere il mercato ma bensì:

"La componente appresa aggiunge valore rispetto a una regola strutturale fissa nella decisione dell'esposizione?"

Per rispondere in maniera rigorosa adottiamo i controlli che nei lavori esaminati non compaiono, spiegati nel dettaglio nei prossimi capitoli.

Limiti

Tutte le conclusioni in questo lavoro valgono all'interno di confini precisi. L'universo di riferimento è quello dei titoli large-cap dell'indice S&P500, cioè le 500 aziende americane con il valore economico più grande. La selezione dei titoli è congelata a equal-weight isolando in questo modo la sola decisione di esposizione. L'obiettivo è il contenimento del drawdown non massimizzare il rendimento. La valutazione è confinata ai periodi e ai regimi di mercato considerati. I risultati vanno quindi letti come un caso illustrativo di un principio generale e non come un verdetto universale sull'intelligenza artificiale in finanza. Su mercati strutturalmente meno efficienti (small-cap, mercati emergenti, crypto-attività), o in problemi dove la selezione cross-asset è determinante, il margine per l'apprendimento potrebbe riaprirsi.

Obiettivo secondario

Oltre all'obiettivo citato di sopra in questo lavoro è presente anche un secondo obiettivo che implementa una componente applicativa. Quest'obiettivo consiste nella realizzazione di un'applicazione per la gestione del portafoglio che integra il modello studiato, rendendolo utilizzabile in modo concreto. La sua architettura e implementazione sono descritte nel dettaglio nel capitolo

Chapter 2

Fondamenti Teorici

In questa sezione vengono fornite al lettore tutte le nozioni di base che servono per comprendere il modello e gli esperimenti.

2.1 Fondamenti economici

2.1.1 Gestione del portafoglio

Gestire il portafoglio significa distribuire il capitale tra un insieme di asset e aggiornarne l'allocazione nel tempo. Ad ogni istante t l'allocazione del portafoglio è un vettore di pesi w_t non negativi che sommano a 1. Gli asset veri e propri sono affiancati da un asset fittizio cash che corrisponde alla liquidità, cioè tutto quel capitale che si decide di non investire. Si definisce esposizione e_t la quota complessiva investita nel paniere di asset veri e propri, mentre la parte restante $(1 - e_t)$ resta in liquidità. Il rendimento del portafoglio in un dato periodo equivale alla media dei rendimenti degli asset pesata per w_t .

Ogni ribilanciamento comporta inoltre costi di transazione in proporzione al volume scambiato. Questi costi includono le commissioni e lo slippage, ovvero la differenza tra il prezzo osservato nel momento in cui si decide di acquistare e il prezzo effettivo a cui avviene la transazione. Modellare questi costi è essenziale per un confronto equo, poiché vanno a penalizzare le strategie con un turnover eccessivo. Il turnover è un termine con cui si indica la frequenza e il volume con cui i vari asset vengono comprati e venduti nel portafoglio in un dato periodo, solitamente misurato su base annuale.

2.1.2 Volatilità e drawdown

La volatilità misura l'ampiezza tipica delle oscillazioni di prezzo attorno alla loro media, quantificando il grado di incertezza e turbolenza a cui è esposto il portafoglio. Cioè prendendo come esempio un titolo che ha un prezzo attuale di 100\$, se esso ha una

volatilità annua del 5% vuol dire che è molto probabile che quel titolo tra un anno avrà un prezzo che oscilla in un range $\pm 5\%$.

A differenza della direzione dei prezzi che è quasi impossibile da prevedere giorno per giorno, la volatilità tende a raggrupparsi nel tempo. Giornate con alta turbolenza sono sistematicamente seguite da altre fasi agitate, e i periodi di calma tendono a persistere nel tempo. Tutto questo rende la volatilità prevedibile.

Questa asimmetria tra la volatilità e la direzione rende economicamente utile agire sul rischio invece che sul rendimento [7].

Il drawdown DD_t è la perdita corrente rispetto al massimo storico del portafoglio. La formula è $(\text{picco} - V_t)/\text{picco}$ e il suo massimo (MDD) è il suo valore peggiore. Esso è la misura di perdita più importante per un investitore ed è l'obiettivo centrale di questo lavoro.

2.1.3 Volatility targeting

Il volatility targeting è una regola classica di gestione del rischio: si scala l'esposizione in modo inversamente proporzionale alla volatilità di mercato, così da tenere il rischio del portafoglio vicino a un livello obiettivo σ_{target} . Nella forma generale l'esposizione è $e_t = (\sigma_{\text{target}}/\sigma_{\text{mkt},t})^p$: con $p = 1$ si ha il volatility targeting classico ($1/\sigma$), Moreira e Muir applicano invece $p = 2$ ($1/\sigma^2$), che riduce l'esposizione in modo più aggressivo quando la volatilità sale. Sempre Moreira e Muir [13] mostrano che gestire l'esposizione sulla volatilità realizzata migliora il profilo rischio/rendimento: è la regola strutturale che fa da riferimento in tutto il lavoro.

2.1.4 Dividendi

Un dividendo è una porzione degli utili aziendali che la società decide di non reinvestire internamente, ma di fornire in contanti ai propri azionisti come pagamento. Il meccanismo funziona in questo modo: nel giorno in cui avviene il dividendo il denaro esce dai bilanci dell'azienda e il prezzo dell'azione in borsa scende di un importo pari al dividendo pagato. Per esempio se un'azione è quotata in borsa a 100\$ e paga 2\$ di dividendo quando riaprirà il mercato sarà quotata a 98\$ e l'azionista si ritroverà 2\$ in liquidità. Quest'azione potrebbe essere un problema durante l'addestramento di un modello in quanto esso registrerà una perdita anche se in realtà non è avvenuta.

2.1.5 Frazionamento azionario (Stock Split)

Un frazionamento avviene quando una società decide di aumentare il numero di azioni in circolazione dividendole, mantenendo però inalterato il valore totale dell'azienda. È come tagliare a metà le fette di una torta già divisa. Per esempio se un'azienda annuncia

uno split di 2 a 1 e tu hai un'azione che vale 100\$, alla riapertura del mercato ti ritroverai con 2 azioni da 50\$. Durante l'addestramento di un agente se i dati non sono aggiustati in base allo split i prezzi di chiusura passerebbero da 100\$ a 50\$ e l'agente penserebbe che sia avvenuto un crollo anche se in realtà non c'è stato.

2.1.6 Regimi di mercato

Bull Market

È una fase prolungata del ciclo economico e finanziario in cui i prezzi degli asset registrano una tendenza costante al rialzo. È caratterizzato da forte ottimismo degli investitori, fiducia nell'economia e da bassa volatilità. In questi periodi le strategie passive come il Buy & Hold ottengono i risultati migliori.

Bear Market

È una fase prolungata di declino dei mercati. Si entra ufficialmente in un bear market quando l'indice azionario registra una perdita superiore al 20% rispetto ai suoi massimi recenti. È caratterizzato da pessimismo, avversione al rischio e da un netto cambio di regime della volatilità, che si alza e rimane elevata in modo strutturale. L'obiettivo principale in questa fase è preservare il capitale.

Crollo

Il crollo è una caduta improvvisa, violenta e verticalizzata dei prezzi che si sviluppa in un arco di tempo che varia da alcuni giorni a poche settimane. È spesso causato da shock sistemici imprevisti o da ondate di vendite dettate dal panico. Genera picchi estremi di turbolenza e volatilità un esempio è il crollo avvenuto durante il Covid.

2.2 Fondamenti Deep Reinforcement Learning

2.2.1 Reinforcement learning

Il Reinforcement learning è un ramo del Machine Learning in cui un sistema software impara a prendere decisioni provando e sbagliando, senza avere qualcuno che gli mostri in anticipo la risposta corretta. Ad ogni passo un agente osserva uno stato s_t , sceglie un'azione a_t secondo una determinata politica e riceve una ricompensa r_t transitando poi allo stato successivo. L'obiettivo è massimizzare il ritorno atteso, cioè la somma scontata delle ricompense future, fattore di sconto $\gamma \in [0, 1)$. La funzione di reward $V(s)$ stima il ritorno atteso a partire da uno stato. Il problema da noi affrontato si addice a questa

formulazione infatti le decisioni di allocazione sono sequenziali, interagiscono con i costi di transazione e hanno effetti nel tempo.

2.2.2 Metodi Actor-Critic e PPO

Gli algoritmi di tipo Policy Gradient ottimizzano direttamente la strategia decisionale dell'agente, senza dover prima stimare il valore numerico di ogni singola azione. Essi però soffrono di una elevata instabilità durante l'addestramento, cioè un'alta varianza del gradiente. Questa caratteristica non gli permette di funzionare a dovere su dati azionari in quanto la maggior parte di essi è costituito da rumore. Per risolvere questo problema si è scelto di adottare un'architettura Actor-Critic, la quale si basa sul dividere il compito dell'apprendimento tra due reti neurali distinte che lavorano insieme:

- **L'Actor:** È la politica vera e propria, osserva il mercato e decide quale azione prendere.
- **Il Critic:** È un valutatore che stima il valore atteso dello stato corrente $V(s_t)$. Cioè prevede quanto il portafoglio guadagnerà in futuro partendo da quella specifica situazione di mercato.

L'aggiornamento dell'Actor si basa sul conteggio di "vantaggio" in formula $A_t = R_t - V(s_t)$, che viene calcolato tramite GAE (Generalized Advantage Estimation). In sostanza l'Actor viene premiato solo se la sua azione ha prodotto un risultato migliore a quello previsto dal Critic, e viene penalizzato se ha fatto peggio. Durante l'addestramento anche il Critic impara a migliorare le sue previsioni.

Per fare in modo che questo processo di addestramento non diverga viene utilizzato l'algoritmo PPO (Proximal Policy Optimization). PPO impedisce che un singolo colpo di fortuna durante l'addestramento, che genera un vantaggio molto grande, vada a cambiare drasticamente i pesi della rete distruggendo la strategia appresa fino a quel punto. Questo viene fatto applicando un meccanismo di taglio alla funzione obiettivo, imponendo che la politica possa aggiornarsi solo a piccoli passi garantendo una convergenza stabile.

2.2.3 Distribuzione beta

Durante l'addestramento, l'agente deve bilanciare lo sfruttamento delle strategie vincenti note con l'esplorazione di nuove possibilità. Per farlo, l'Actor non restituisce un'azione deterministica fissa, ma campiona la sua decisione da una distribuzione di probabilità.

In questo lavoro è stata usata la distribuzione Beta. La distribuzione Beta è una distribuzione di probabilità continua definita nell'intervallo chiuso $[0, 1]$. Questa caratteristica la rende adatta a questo lavoro, poiché non potrà mai suggerire azioni non ammesse per l'obiettivo, come investire il 120% del capitale. La forma della curva non è fissa ma

viene modificata da due parametri positivi α e β . In questo lavoro la Beta campiona il moltiplicatore di esposizione m_θ , che attraverso la formula fissa di volatility targeting decide l'esposizione al mercato e_t ; regolare α e β significa impostare quanto la rete decide di esporsi. Quando la rete sbilancia la curva verso 1 ($\alpha > \beta$) sta imparando che in quel regime di mercato conviene un m_θ alto, ovvero un'esposizione elevata. Quando la sposta verso 0, impara a ridurre l'esposizione mantenendo più liquidità. Se la rete apprende valori molto alti per entrambi i parametri, la curva si concentra in un picco stretto, vuol dire che l'agente ha smesso di esplorare ed è sicuro dell'esposizione da scegliere. Se invece la curva è piatta $\alpha = \beta = 1$, l'agente non sa cosa fare e prova a caso qualsiasi valore tra 0 e 1.

2.2.4 Reti ricorrenti e invarianza di dominio

Le serie storiche finanziarie sono elaborate tramite una rete ricorrente GRU (Gated Recurrent Unit). La (GRU) [5] è un'architettura ricorrente progettata per modellare dipendenze temporali complesse su sequenze arbitrarie, mitigando il problema della scomparsa del gradiente. A differenza delle reti ricorrenti standard o delle LSTM, la GRU sintetizza l'intera memoria storica in un unico stato latente h_t , regolandone il flusso mediante due meccanismi a porta con attivazione sigmoidea:

1. **Reset Gate (r_t):** definita come $r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$, determina in quale misura la memoria passata h_{t-1} debba essere ignorata nel calcolo della nuova rappresentazione candidata $\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$;
2. **Update Gate (z_t):** definita come $z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$, controlla quanta informazione del precedente stato h_{t-1} debba essere preservata rispetto alla nuova proposta $\tilde{h}_t$.

Lo stato nascosto finale è aggiornato tramite interpolazione lineare:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \quad (2.1)$$

Questa formulazione permette all'unità di conservare intatte caratteristiche rilevanti lungo finestre temporali estese oppure di resettare rapidamente il contesto in presenza di discontinuità o cambi di regime nella serie.

2.2.5 Apprendimento supervisionato

L'apprendimento supervisionato è un modello del machine learning in cui una rete neurale o statistica viene addestrata un set di dati etichettati. L'algoritmo riceve in input sia i dati di partenza che le risposte corrette corrispondenti, ovvero le etichette. L'obiettivo dell'addestramento è costruire una funzione matematica in grado di mappare correttamente gli input negli output. E lo si fa cercando di minimizzare mano a mano l'errore tra le

previsioni della rete e le etichette. Una volta terminato il training il modello ha imparato a generalizzare riuscendo a dare la risposta corretta anche su dati nuovi e mai osservati.

2.2.6 Binary Cross-Entropy Loss (BCE)

La BCE è una specifica loss function utilizzata per ottimizzare i modelli neurali nei problemi di classificazione binaria, cioè quando la variabile target ha solo due valori possibili. Questa funzione calcola la distanza tra la probabilità predetta dal modello e l'etichetta nel dataset. La sua caratteristica distintiva è l'utilizzo del logaritmo che applica una penalità esponenziale agli errori commessi con elevata confidenza. Se la rete neurale è estremamente sicura di una classificazione errata, il termine logaritmico esplode verso l'infinito, producendo un errore altissimo che forza l'ottimizzatore a correggere drasticamente i pesi durante la backpropagation.

Chapter 3

Progettazione dataset e metodologia sperimentale

3.1 Progettazione del dataset

3.1.1 Fonte e universo

I dati di mercato sono stati scaricati da Yahoo Finance tramite la libreria `yfinance` con l'opzione `auto_adjust=True`. Questa opzione fornisce i dati già aggiustati per split e dividendi su base giornaliera in modo da evitare che il modello percepisca delle crisi quando in realtà è solo l'effetto causato da questi ultimi. Nel dataset sono presenti titoli large-cap del mercato americano, cioè titoli che rappresentano un mercato molto liquido ed efficiente. Nello specifico i titoli nel dataset costituiscono l'S&P500, un indice azionario che raggruppa le 500 aziende a maggiore capitalizzazione e rilevanza economica degli Stati Uniti. Il dataset contiene uno storico di questi titoli che va dal 2000 al 2026. Per ogni titolo sono presenti su base giornaliera quattro valori:

1. **high**: massimo prezzo raggiunto dall'asset nella giornata
2. **low**: minimo prezzo raggiunto dall'asset nella giornata
3. **close**: prezzo di chiusura della giornata per quell'asset
4. **vix**: indice di paura del mercato condiviso fra tutti i titoli

È importante specificare che il dataset è composto solo dalle aziende che fanno parte attualmente dell'S&P500. Non compaiono quindi tutte quelle società che sono uscite dall'indice per fallimento dal 2000 a oggi. Questo è un limite accettato in quanto l'obiettivo non è la valutazione dei rendimenti ma un confronto fra strategie che operano sullo stesso paniere.

3.1.2 Feature e loro selezione

La scelta di avere nel dataset solo questi quattro indicatori si è consolidata man mano che il modello veniva testato. All'inizio si era partiti da un grande insieme di indicatori tecnici generati tramite la libreria di FinRL; successivamente, attraverso varie ablation, si è arrivati a questo insieme minimale. Infatti l'ablation ha dimostrato che la decisione è dominata dalla volatilità e che la maggior parte di questa moltitudine di indicatori non aggiungeva alcuna informazione utile a stimarla con precisione più di quanto non facessero i 4 indicatori definitivi.

3.1.3 Suddivisione dei dati

La suddivisione dei dati per training, validation e test set è temporale. Questo per evitare che durante la simulazione di trading il modello abbia accesso a dei dati futuri rispetto al momento simulato in cui si trova.

Insieme	Periodo	Uso
Training	2000-01-01 – 2012-01-01	stima dei parametri della rete
Validazione	2012-01-01 – 2015-01-01	selezione/controllo del modello
Test (out-of-sample)	2015-01-01 – 2026-02-01	valutazione finale

Table 3.1: Suddivisione temporale del dataset.

Tutti i risultati riportati si riferiscono al periodo di test, mai osservato durante l'addestramento. I panieri operativi per gli esperimenti sul valore della parte appresa sono composti da 10 titoli per universo. Per la parte di test sulla generalità sono stati usati panieri che si differenziano tra loro per caratteristiche e numero di asset.

3.2 Metodologia sperimentale

3.2.1 Strategia appresa e formula di riferimento

La decisione del modello è l'esposizione del capitale al mercato. La quota che sceglie di non tenere in liquidità viene poi ripartita equamente tra i titoli del paniere fornito in input. L'esposizione si basa su un overlay di volatility targeting:

$$e_t = m_\theta \cdot \underbrace{\left(\frac{\sigma_{\text{target}}}{\sigma_{\text{mkt},t}} \right)^p}_{\text{volatility targeting}},$$

dove $\sigma_{\text{mkt},t}$ è la volatilità realizzata del mercato, σ_{target} la volatilità obiettivo, m_θ il moltiplicatore appreso e p l'esponente. La formula fissa ($m_\theta = 1$, p costante) è il riferimento non appreso rispetto al quale si misura ogni componente adattiva.

3.2.2 Funzione di reward

L'addestramento del modello tramite PPO ha l'obiettivo di massimizzare una reward che unisce rendimento e controllo del rischio. La reward usata **DD_VOL_PENALTY** è il rendimento logaritmico del portafoglio al quale vengono sottratte due penalità:

$$r_t = \underbrace{\ln \frac{V_t}{V_{t-1}}}_{\text{crescita}} - \underbrace{\lambda_{dd} DD_t^\alpha}_{\text{penalità drawdown}} - \underbrace{\lambda_{vol} e_t \sigma_{\text{mkt},t}}_{\text{penalità esposizione} \times \text{vol}}. \quad (3.1)$$

Il primo termine è il rendimento logaritmico dello step, che viene sommato a quello precedente durante tutto l'episodio e insieme formano la crescita logaritmica totale del capitale. Il secondo termine penalizza il drawdown dello step corrente DD_t elevato a un esponente $\alpha = 1, 5$. Il valore di α è stato scelto in modo da far crescere la penalità man mano che il drawdown si aggrava, dando poca urgenza alle perdite superficiali e molta a quelle profonde. Il terzo termine penalizza l'esposizione e_t in proporzione alla volatilità di mercato, rendendo esplicito il legame rischio/esposizione e facendo in modo che il modello non debba scoprire per tentativi che è da evitare una grande esposizione nei periodi di alta volatilità. I pesi usati sono $\lambda_{dd} = 0.01$ e $\lambda_{vol} = 0.002$.

3.2.3 Protocollo di valutazione

Ogni modello è valutato con un backtest statico sul periodo di test. I pesi vengono ricalcolati a ogni passo applicando i costi di transazione. I backtest sono stati fatti su periodi che rappresentano vari regimi di mercato:

Regime	Periodo	Caratterizzazione
Test completo	2015-01-01 – 2026-02-01	campione out-of-sample lungo
2017	2017-01-01 – 2017-12-31	mercato calmo/rialzista
COVID	2020-02-01 – 2020-08-31	crash rapido e rimbalzo
2022	2022-01-01 – 2022-12-31	ribasso lento e prolungato
2025	2025-02-01 – 2026-02-01	sell-off dei dazi (apr 2025)

Table 3.2: Regimi di valutazione.

Per ogni configurazione si addestrano più seed (42, 43, 44) e si riporta la mediana tra i seed. Questo perché gli esperimenti preliminari hanno mostrato che seed diversi possono dare esiti sensibilmente diversi, quindi la robustezza al seed è parte del protocollo.

Confronto a parità di cash medio. Per verificare rigorosamente che la parte appresa del modello abbia davvero aggiunto valore è stato costruita una frontiera di riferimento. La frontiera è stata costruita partendo dal volatility targeting di Moreira e Muir e tracciando una curva di punti (cash medio, MDD) al variare del target di volatilità σ_{target} . Sono state poi confrontate le performance dell'agente con la frontiera per capire se esso si è riuscito a posizionare in un'area del grafico più conveniente rispetto alla frontiera.

3.2.4 Baseline di confronto

Ogni strategia appresa è confrontata con riferimenti non appresi:

- la **formula fissa** ($m_\theta = 1$, p costante), per isolare il contributo dell'apprendimento rispetto alla sola struttura;
- il **paniere equipesato** pienamente investito, per avere un riferimento senza gestione dell'esposizione in modo da comprendere quanto l'overlay riduce il drawdown rispetto a restare sempre esposti;
- la posizione **100% cash**, come null di riferimento per il controllo del drawdown.

3.2.5 Metriche di valutazione

Essendo l'obiettivo la riduzione del drawdown a parità di capacità di stare sul mercato, si adotta il seguente insieme di metriche.

Metriche primarie.

- **Maximum Drawdown (MDD)**: massima perdita dal picco, $\max_t(\text{picco}_t - V_t)/\text{picco}_t$. È la metrica guida.
- **Calmar**: CAGR/MDD. Rendimento per unità di drawdown. Premia chi riduce il drawdown senza rinunciare al rendimento. Serve a capire se la difesa è ottenuta semplicemente restando in liquidità o da una strategia di esposizione intelligente.
- **Sharpe**: rendimento medio / volatilità (annualizzato). Utile in contrasto col Calmar per distinguere il controllo del rischio dalla sola conservatività.
- **Cash medio**: capitale medio tenuto in liquidità. Rende esplicito il costo di ogni riduzione di drawdown: un MDD più basso ottenuto con più cash è conservatività, non tempismo.

Metriche per il confronto sui panieri. Per valutare la selezione rispetto all'equipesato si usano inoltre:

- **MDD_gain**: riduzione di drawdown rispetto a un null a liquidità costante pari al cash medio del modello (> 0 indica difesa attiva reale);
- **correlazione cash-drawdown**: verifica che la liquidità sia alzata in prossimità dei ribassi e non in modo indiscriminato;
- **up/down capture**: quota di rialzo catturata e di ribasso subita rispetto al mercato.

Chapter 4

Implementazione modello

4.1 Modello

4.1.1 Introduzione

Il modello implementato rappresenta una variante dell'architettura proposta da Carlos Bentancour e Wen-Hui Chen nell'articolo "Deep reinforcement learning for portfolio management of markets with a dynamic number of assets" [2] adattata al mercato azionario "S&P500". L'innovazione principale di questa architettura risiede nella proprietà DNA (Dynamic Number of Assets), ovvero una struttura che permette di operare su mercati finanziari in cui il numero di asset varia nel tempo. Questa caratteristica ha determinato la scelta dell'architettura per l'implementazione on-device: essa consente infatti all'utente di selezionare un paniere arbitrario di titoli e calcolarne l'allocazione ottimale dei pesi in fase di inferenza, senza richiedere una fase di riaddestramento specifico sui singoli asset scelti.

4.1.2 Feature

Le feature date impasto al modello sono di 3 tipi diversi

Feature per asset

Sono le uniche feature che passano dalla GRU. Vengono passate per ogni asset e per ogni giorno della sliding window, sono **high**, **low**, **close**, **VIX**. Le prime tre servono per leggere la volatilità e il trend recente di ogni titolo. VIX è un dato di mercato, è identico per tutti i titoli nella stessa data, passa sia dalla GRU ma anche come scalare globale in **risk_state** arrivando alle teste sia filtrato sia grezzo. Forma: $(B, N, 30, 4)$ dove B è la dimensione del batch, N è il numero di asset nel paniere, 30 la dimensione della finestra scorrevole e 4 le features in input.

Normalizzazione

Ogni osservazione è una finestra scorrevole di $W = 30$ giorni di contrattazione. Le tre serie storiche di prezzo (high, low, close) vengono normalizzate dividendo ciascun valore per l'ultimo prezzo di chiusura della finestra di osservazione in modo da renderli dei rapporti puri. La normalizzazione è necessaria per tre motivi. In primo luogo serve perché i livelli di prezzo assoluti crescono di ordini di grandezza tra il periodo di addestramento e quello di test, se non normalizzassimo gli input essi risulterebbero fuori scala. In secondo essa serve a mantenere l'equità dell'aggregazione, dato che il contesto di mercato è la media permutation-invariant degli embedding per-asset se tenessimo i prezzi grezzi essi andrebbero a distorcere il segnale di esposizione. Infine serve perché la valutazione su panieri diversi da quello di addestramento richiede input privi di unità di misura.

Il VIX è riscalato a scala fissa (/100), questa normalizzazione a scala fissa evita di introdurre informazione dipendente dal futuro, poiché non viene stimata alcuna media o deviazione sull'intero campione.

prev_weights (B, N)

La composizione attuale del portafoglio prima di agire: una percentuale per ogni asset che somma a 1. Alla rete però viene data come feature solo la quota CASH attuale in quanto la selezione dei titoli normali è sempre equal-weight.

risk_state (B, 6)

Sei scalari globali (identici per ogni asset) che indicano lo stato di rischio dell'agente (i primi 3) e del mercato nel suo insieme.

1. **Drawdown corrente:** Misura la perdita percentuale rispetto al massimo storico del capitale nell'episodio. Serve all'agente per capire quanto sta perdendo rispetto al picco inducendo a ridurre l'esposizione se la strategia sta soffrendo.

- *Trasformazione:* $\frac{\text{picco} - \text{valore}}{\text{picco}}$
- *Range:* [0.0, 1.0]

2. **Volatilità del portafoglio:** deviazione standard annualizzata dei rendimenti del portafoglio a 21 giorni (un mese), misura il rischio specifico delle scelte di allocazione correnti dell'agente.

- *Trasformazione:* $\ln(1 + \sigma_{ann})$
- *Range:* [0.0, 1.0]

3. **Tempo sott'acqua:** numero di giorni consecutivi senza raggiungere un nuovo picco del capitale, aiuta l'agente a capire se il portafoglio è bloccato in una fase di stallo prolungata o se recupera velocemente

- *Trasformazione:* $\ln(1 + \frac{giorni}{252})$
- *Range:* $[0.0, 1.5]$

4. **Volatilità di mercato compressa:** volatilità storica a 21 giorni del paniere (tutti i titoli), da il contesto macroeconomico permettendo di distinguere tra mercati calmi e mercati volatili

- *Trasformazione:* $\ln(1 + \sigma_{mkt_ann})$
- *Range:* $[0.0, 0.8]$

5. **Livello del vix:** l'indice VIX della volatilità attesa a 30 giorni, fornisce una misura di aspettativa futura

- *Trasformazione:* $\frac{VIX}{100}$
- *Range:* $[0.1, 0.8]$

6. **Volatilità di mercato grezza:** uguale all'altra ma non in scala logaritmica, permette al modello di percepire direttamente l'intensità degli shock del mercato senza smorzarli.

- *Trasformazione:* $\min(\max(\sigma_{mkt_ann}, 0), 3.0)$
- *Range:* $[0.0, 3.0]$

4.1.3 Architettura

L'intera architettura è stata progettata per funzionare con un numero qualsiasi di asset, in qualsiasi ordine. Questo risultato è ottenuto con due proprietà:

- **Permutation-equivariance:** la stessa rete è applicata ad ogni asset indipendentemente (i pesi sono condivisi), cioè scambiare due asset scambia solo i loro output
- **Permutation-invariance:** il contesto di mercato si ottiene con una media sugli asset, non dipende da quanti sono nè da come sono ordinati

Per convenzione il CASH cioè la liquidità è sempre l'ultimo asset lungo l'asse 1.

Rete

La rete del modello presenta un'architettura Actor-Critic. Ogni giorno comprime lo storico di ogni asset con una GRU condivisa facendone la media in modo da ottenere un contesto di mercato indipendente dal numero di asset. Da lì produce la sua decisione indicando quanto esporsi al mercato, ovvero gestisce quanto parte del portafoglio viene investita (equal-weight per gli asset) e quanta parte rimane in contanti. La rete si divide in sei componenti fondamentali, riassunti nel diagramma di Fig. 4.1.

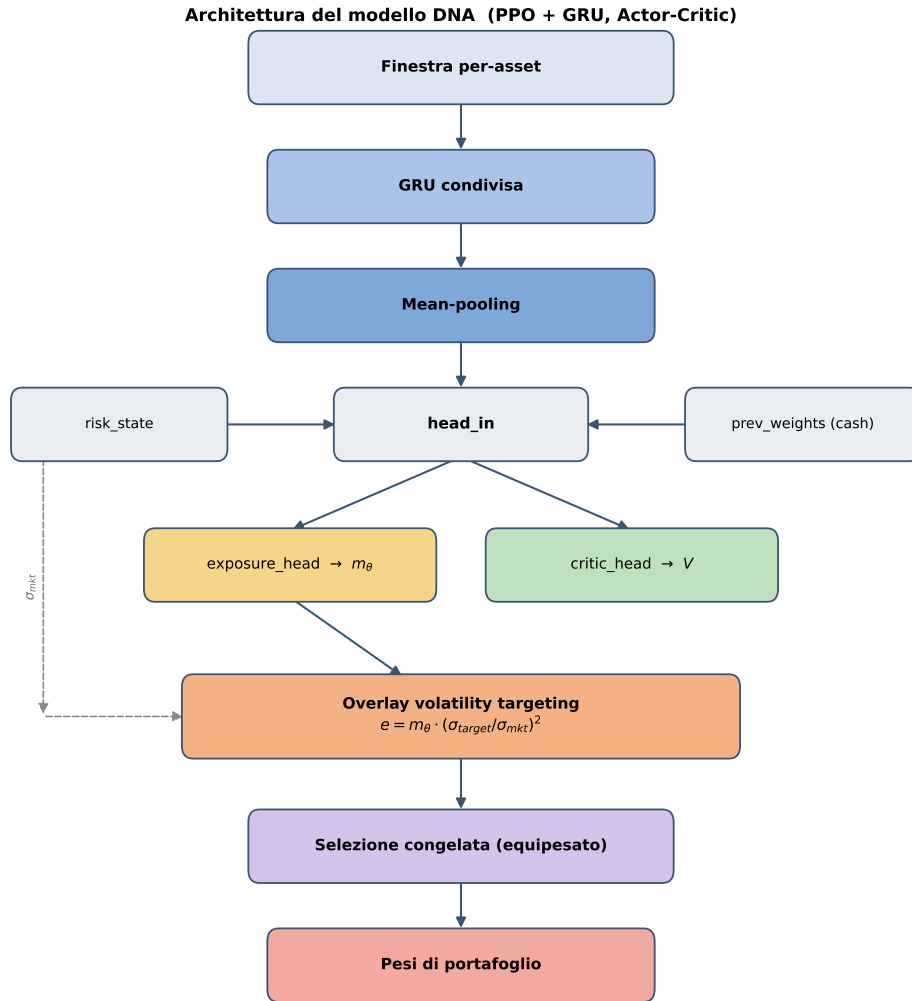


Figure 4.1: Architettura del modello: dalla finestra per-asset, tramite GRU condivisa e mean-pooling, al vettore `head_in` (con `risk_state` e `prev_weights`), fino all'overlay di volatility targeting e ai pesi di portafoglio.

Prima di effettuare qualunque operazione avviene la fase di normalizzazione descritta poco sopra, in seguito questi dati vengono passati alla GRU.

GRU Riceve features della forma $(B, N, 30, 4)$ e applica un reshape che fonde **B** e **N** in un'unica dimensione di taglia $B \times N$, usata come nuovo batch della GRU. In questo modo ogni asset di ogni campione diventa un esempio indipendente processato dalla stessa identica GRU: in sostanza c'è una sola GRU applicata a ciascuno degli N asset. La struttura interna della GRU è composta da 2 layer impilati, `hidden_size=10`, `batch_first=True`.

- Layer 1: input 4 feature $\rightarrow$ 10 hidden.
- Layer 2: 10 $\rightarrow$ 10.

Si prende in seguito solo l'ultimo timestep ovvero un vettore di 10 elementi contenente una sintesi appresa della dinamica recente dell'asset. Ora abbiamo un embedding per ogni asset, poiché la stessa rete è applicata a ciascun asset in modo indipendente, l'embedding di ciascuno dipende solo dalla sua serie storica, non da chi gli sta accanto nè dalla sua posizione nella lista. Quindi scambiando due asset in input gli embedding si spostano insieme ai loro asset. È ancora presente l'equivarianza. Dopo aver ottenuto gli embedding ne calcoliamo la media per ottenere il contesto di mercato.

```
1 context = last_hidden.mean(dim=1)
```

La media è un'operazione simmetrica quindi il `context` sarà identico qualunque sia l'ordine degli asset in questo modo otteniamo l'invarianza. L'invarianza ci permette di utilizzare il modello su qualsiasi asset non solo quelli su cui è stato addestrato. Anche se la media ci fa perdere la capacità di distinguere i singoli asset possiamo usarla in quanto, decidere quanto esporsi al mercato è una proprietà del portafoglio nel suo insieme e non deve dipendere da un'ordinamento arbitrario

Actor Nel modello l'Actor non si occupa di scegliere direttamente l'esposizione, ma produce un unico moltiplicatore appreso m_θ che andrà poi a correggere una formula fissa di volatility targeting. Riceve in input un vettore da 17 elementi, ottenuto concatenando il contesto di mercato (i 10 numeri in output della GRU), il `risk_state` del portafoglio e la quota di cash attuale:

Listing 4.1: Struttura della testa di esposizione

```
1 exposure_head = Sequential(
2     Linear(17 -> 16), ReLU, Linear(16 -> 1)
3 )
4 m_theta = softplus(exposure_head(input)) # m_theta > 0
```

L'output della exposure head passa poi in una `softplus` che garantisce $m_\theta > 0$. Il bias dell'ultimo layer è inizializzato in modo che al passo zero `softplus(bias) = 1`. Quindi il modello parte con $m_\theta \approx 1$, cioè come volatility targeting puro. I pesi del layer sono inoltre

scalati di un fattore 0.1 in questo modo la parte da apprendere cresce lentamente invece di dominare da subito con valori casuali. In sostanza l'unico grado di libertà appreso della decisione è una correzione moltiplicativa attorno a 1 sopra una regola deterministica.

Overlay di volatility targeting È il componente che trasforma il moltiplicatore appreso m_θ nella decisione di esposizione e_t , applicando la formula di volatility targeting descritta nella metodologia (Sez. 3.2). Una volta ottenuta e_t la selezione degli asset è congelata a equal weight e la liquidità riceve $1 - e_t$. L'ordine con cui elenchiamo i titoli non conta. Infatti se invertiamo la posizione di due asset all'ingresso del modello, anche i rispettivi pesi in uscita si scambieranno di posto, senza alterare la decisione finale.

Critic Il Critic serve unicamente durante l'addestramento come baseline per il calcolo del vantaggio in PPO. Fornisce all'algoritmo PPO una stima del valore dello stato che serve a rendere l'apprendimento più stabile. Ha la stessa struttura della `exposure_head` e riceve lo stesso identico input da 17 elementi:

Listing 4.2: Struttura del critic

```

1 critic_head = Sequential(
2     Linear(17 -> 16), ReLU, Linear(16 -> 1)
3 )

```

L'output è un singolo scalare $V(s)$ che rappresenta la stima del ritorno atteso a partire dallo stato corrente. Questo valore viene usato da PPO come baseline nel calcolo del vantaggio: invece di aggiornare la policy in base al ritorno grezzo R_t si usa infatti il vantaggio $A_t = R_t - V(s_t)$ che misura quanto meglio del previsto è andata l'azione. Sottrarre $V(s_t)$ riduce la varianza del gradiente e quindi rende l'addestramento più stabile e veloce.

Output del modello Il modello restituisce in output il vettore dei pesi di portafoglio

$$w = \left[\underbrace{e_t/(N-1), \dots, e_t/(N-1)}_{N-1 \text{ asset rischiosi}}, \underbrace{1-e_t}_{\text{cash}} \right],$$

dove e_t è l'esposizione prodotta dall'overlay vol targeting. È una distribuzione che somma a 1, con ultimo elemento la liquidità e gli asset tutti allo stesso peso.

4.1.4 Environment

L'ambiente `PortfolioEnv` implementa la simulazione di mercato con cui l'agente interagisce. A ogni step (un giorno di borsa) gli fornisce un'osservazione, riceve un'azione, esegue il ribilanciamento tenendo conto dei costi, fa evolvere il valore del portafoglio e calcola la ricompensa.

Lo stato che l'agente riceve a ogni step è composto da tre elementi:

- **feature di mercato** per ogni asset (la finestra di 30 giorni di `high`, `low`, `close`, `vix`), che passano per la GRU;
- **pesi correnti** del portafoglio `prev_weights`;
- **stato di rischio** `risk_state`, sei scalari globali (uguali per tutti gli asset) che descrivono la salute del portafoglio e il regime di mercato.

La volatilità e il tempo in drawdown sono compresse con $\log(1 + x)$ per fare in modo che pochi eventi estremi non possano pesare troppo nell'osservazione, il drawdown resta lineare.

Una volta ricevuti i nuovi `prev_weights`, l'ambiente li applica attraverso due passaggi. Prima calcola i costi di transazione legati al ribilanciamento, poi aggiorna il valore del portafoglio applicando i prezzi di mercato del giorno corrente. Da questo ricalcolo si ricavano il rendimento giornaliero, il nuovo massimo storico raggiunto e il drawdown attuale. Infine, la volatilità di mercato impiegata nel `risk_state` e nell'overlay non è una variabile esterna fornita dal dataset, ma un segnale calcolato internamente. Essa è la deviazione standard mobile dei rendimenti a 21 giorni del paniere equipesato, riscalata su base annua ($\times \sqrt{252}$).

Costi di transazione (TransactionOptimizer) Nella realtà ogni operazione di mercato prevede il pagamento di commissioni. Passare dalla composizione attuale del portafoglio w' ai nuovi pesi desiderati w richieda di comprare e vendere titoli. Anche nell'ambiente simulato è stato fatto in modo che per ogni transazione si paghi una commissione dello 0,2%. Questo costo viene quantificato tramite un fattore di rimanenza $\mu \in (0, 1]$, che rappresenta la quota di capitale effettivamente intatta dopo aver pagato i costi: $V_{\text{post}} = \mu V_{\text{pre}}$. Il valore di μ è definito dall'equazione:

$$\mu = 1 - \sum_i \left[b_i \max(\mu p_i - p'_i, 0) + s_i \max(p'_i - \mu p_i, 0) \right], \quad (4.1)$$

dove p_i e p'_i sono le quote target e correnti del titolo i , mentre b_i e s_i indicano le commissioni per acquisti e vendite. Il capitale finale μ dipende dal ricavo ottenuto vendendo i vari titoli a cui vengono sottratti i costi di commissione, l'ammontare delle commissioni però dipende a sua volta dal capitale finale. Questo problema è stato risolto tramite una serie di iterazioni a punto fisso adottando lo schema proposto da Jiang et al. [10]. Anche nel modello originale della DNA [2] sono applicati i costi di transazione, ma vengono calcolati attraverso un programma lineare (LP). Si è scelto di implementare i costi di transazione con un metodo di iterazione a punto fisso perché in questo modo è stato possibile trasferire il modello on-device nell'applicazione. Includere questi costi impedisce alla rete di ottenere

facili guadagni teorici attraverso un continuo ribilanciamento e garantisce che il risultato sia il più simile possibile a una situazione reale.

Episodio e ricompensa Un episodio non ha lunghezza fissa, esso parte da un giorno estratto a caso e prosegue fino all'ultimo giorno del periodo di addestramento. La durata varia quindi da un minimo garantito di 252 giorni di borsa (circa un anno) fino all'intero orizzonte di training. A ogni passo l'ambiente restituisce la ricompensa, la cui forma è discussa nella sezione sulla metodologia (Sez. 3.2).

Provenienza Il termine base di log-rendimento è la scelta standard nel portfolio management con reinforcement learning. I due termini di penalità non derivano invece da un singolo lavoro di letteratura, ma sono un reward shaping progettato appositamente per questo modello: sono la traduzione, nella funzione obiettivo, dell'idea che la difesa debba attivarsi quando il rischio di mercato aumenta. L'overlay su cui la reward agisce è a sua volta l'analogo appreso del volatility targeting [13].

4.1.5 Addestramento

Il modello è stato addestrato con **Proximal Policy Optimizaton** (PPO), un algoritmo di reinforcement learning on-policy della famiglia Actor-Critic [15]. L'addestramento alterna una fase di raccolta di esperienza ottenuta facendo interagire la policy corrente con l'ambiente, con il successivo aggiornamento dei pesi in base ai dati raccolti.

Algoritmo di addestramento (PPO) L'addestramento segue uno schema iterativo suddiviso in tre fasi:

1. **Raccolta dell'esperienza:** a ogni ciclo, l'agente interagisce simultaneamente con `n_envs` ambienti di mercato indipendenti, registrando una traiettoria di `n_steps` passi temporali per ciascuno.
2. **Valutazione delle decisioni (Advantage):** il vantaggio A_t di ogni azione è stimato con la Generalized Advantage Estimation (GAE) [14], ovvero lo scarto tra il rendimento ottenuto e quello previsto dal Critic (cfr. Cap. 2).
3. **Aggiornamento prudente della policy:** i pesi sono aggiornati massimizzando l'obiettivo clippato di PPO, che limita l'ampiezza di ogni singolo aggiornamento e la divergenza della strategia (cfr. Cap. 2).

La funzione di perdita complessiva combina la loss della policy con l'errore di stima del Critic (pesato dal coefficiente `vf_coef`). L'ottimizzazione viene eseguita con l'algoritmo Adam su minibatch, applicando un vincolo sul gradiente per impedire oscillazioni numeriche distruttive lungo le `n_epochs` passate sui dati. La catena che lega il tutto è:

la reward si accumula nel ritorno R_t ; il Critic prova a prevederlo con $V(s)$; la differenza è il vantaggio; il vantaggio entra nella loss che spinge l'Actor a ripetere le azioni con vantaggio positivo.

Esplorazione Dato che la selezione è congelata e l'unica decisione appresa è l'esposizione, PPO esplora campionando solo l'esposizione da una distribuzione Beta. La rete produce un valore di esposizione deterministico, la policy è la Beta centrata su quel valore, che permette all'agente di esplorare valori di esposizione vicini. L'ampiezza dell'esplorazione è regolata da un parametro appreso κ , così la rete decide da sola quanto esplorare. In inferenza il campionamento è rimosso: la policy diventa deterministica e coincide con l'esposizione prodotta dalla rete.

Dati e suddivisione temporale L'addestramento avviene su un paniere di 8 large-cap (CAT DIS HD IBM JNJ JPM PG UNH) con finestra di osservazione di 30 giorni. Sono stati scelti solo 8 titoli perché esperimenti preliminari hanno dimostrato che usare un paniere più ampio durante l'addestramento non migliora le performance ma rende il modello meno cauto nella gestione dell'esposizione. I dati sono divisi cronologicamente per evitare qualsiasi lookahead: addestramento sul periodo 2000–2012, validazione 2012–2015, e test fuori campione dal 2015 in poi. Grazie all'invarianza per permutazione e all'N-agnosticismo, il modello allenato su questo paniere può poi essere valutato zero-shot su universi diversi.

Iperparametri

Iperparametro	Valore
Ambienti paralleli (n_envs)	8
Passi per rollout (n_steps)	128
Epoche per aggiornamento (n_epochs)	5
Dimensione minibatch	256
Passi totali di training	700 000
Fattore di sconto γ	0.9
λ della GAE	0.95
Raggio di clipping ϵ	0.2
Peso value loss (vf_coef)	0.5
Coeff. entropia (ent_coef)	0
Learning rate (Adam)	3×10^{-4}
Norma massima del gradiente	0.5
Concentrazione iniziale κ	50

Chapter 5

Esperimenti e Risultati

5.1 Conclusioni: cosa aggiunge l'apprendimento

In questa parte andiamo a esporre i vari esperimenti fatti, mostrando i risultati ottenuti da questo modello.

5.1.1 Esperimento 1: Come influisce il parametro σ_{target}

Congelando la parte appresa ($m_\theta = 1$, Moreira–Muir pura) il profilo di rischio non cambia. Sul periodo 2015/26 il modello appreso e la formula pura hanno lo stesso massimo drawdown entro il rumore (MDD 0.185 vs 0.219) e un Calmar confrontabile (1.24 vs 1.13); la rete converge sistematicamente su $m_\theta \approx 1$ riproducendo in sostanza la regola deterministica.

Confrontando varie versioni del modello che differiscono per il parametro σ_{target} , cioè la volatilità annuale del portafoglio che il modello cerca di mantenere, capiamo che cambiare questa variabile non porta a dei miglioramenti. σ_{target} è la manopola che abbiamo sul drawdown, aumentando il suo valore aumenta anche il massimo drawdown ottenuto in modo monotono, insomma è il parametro che ci permette di scegliere il rischio di perdita che vogliamo avere e non porta a nessun tipo di ottimizzazione dell'apprendimento.

Table 5.1: Esperimento 1 — overlay appreso (m_θ) vs formula pura ($m_\theta=1$, Moreira–Muir) ai vari σ_{target} , periodo 2015–2026 (mediana su 3 seed).

σ_{target}	Overlay	cash	MDD	Calmar	Sharpe
0.10	appreso (m_θ)	0.790	0.056	0.85	1.115
	formula ($m_\theta=1$)	0.615	0.089	1.04	1.240
0.15	appreso (m_θ)	0.438	0.105	1.35	1.361
	formula ($m_\theta=1$)	0.337	0.138	1.35	1.442
0.20	appreso (m_θ)	0.217	0.185	1.24	1.466
	formula ($m_\theta=1$)	0.179	0.219	1.13	1.471

Conservatività, non intelligenza. Per capire se la rete neurale abbia davvero sviluppato una capacità di prevedere quando uscire dal mercato prima dei crolli, è necessario separare l'effetto del tempismo da quello del livello medio di liquidità mantenuto. Per farlo abbiamo tracciato una frontiera di controllo. Abbiamo preso un piano cartesiano dove l'asse x contiene il livello di cash medio mantenuto mentre l'asse y contiene l'MDD. Su questo piano cartesiano viene tracciata una curva che unisce le varie collocazioni della formula di volatility targeting di Moreira e Muir al variare di σ_{target} . Se l'agente avesse appreso un'intelligenza predittiva superiore, dovrebbe posizionarsi al di sotto di questa curva: a parità di liquidità media, dovrebbe cioè subire perdite massime inferiori grazie alla tempestività dei suoi interventi. (Fig. 5.1).

I dati mostrano invece l'esatto contrario: **il modello appreso cade esattamente sulla frontiera della formula deterministica** in tutti i cinque regimi di mercato considerati. Gli scarti di MDD sono trascurabili e privi di un pattern sistematico. Questo dimostra che il drawdown minore ottenuto dal modello rispetto alla formula standard allo stesso σ_{target} non deriva da una gestione dinamica intelligente, ma semplicemente dal fatto che l'agente sceglie di rimanere mediamente più disinvestito. Si tratta di pura conservatività: lo stesso identico livello di protezione si può ottenere con la formula statica, limitandosi ad abbassare il bersaglio di volatilità.

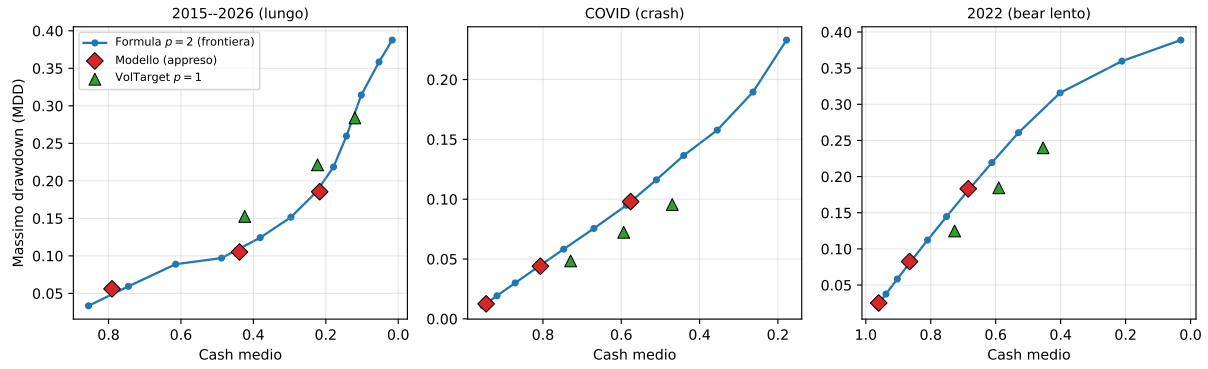


Figure 5.1: Frontiera cash medio–MDD. Curva blu: formula fissa ($p = 2$, $m_\theta = 1$) al variare di σ_{target} ; rombi rossi: modello appreso ($\sigma_{\text{target}} = 0.10, 0.15, 0.20$); triangoli verdi: volatility targeting classico ($p = 1$), stessi σ_{target} .

Table 5.2: Confronto a $\sigma_{\text{target}} = 0.20$ delle quattro strategie (mediana su 3 seed). “modello” = m_θ appreso; “formula” = Moreira–Muir pura ($p = 2$); “voltarget” = volatility targeting classico ($p = 1$); “equipesato” = paniere equipesato pienamente investito.

Periodo	Strategia	MDD	Calmar	Sharpe
2015–26	modello	0.185	1.24	1.47
	formula	0.219	1.13	1.47
	voltarget	0.284	0.96	1.44
	equipesato	0.379	0.80	1.25
COVID	modello	0.098	4.17	2.47
	formula	0.116	4.05	2.47
	voltarget	0.096	7.20	2.83
	equipesato	0.323	2.99	1.71
2022	modello	0.183	−0.95	−1.62
	formula	0.219	−0.95	−1.60
	voltarget	0.240	−0.91	−1.13
	equipesato	0.379	−0.95	−1.15
2025	modello	0.153	0.94	0.98
	formula	0.165	0.92	0.98
	voltarget	0.100	3.54	1.95
	equipesato	0.227	1.13	1.08

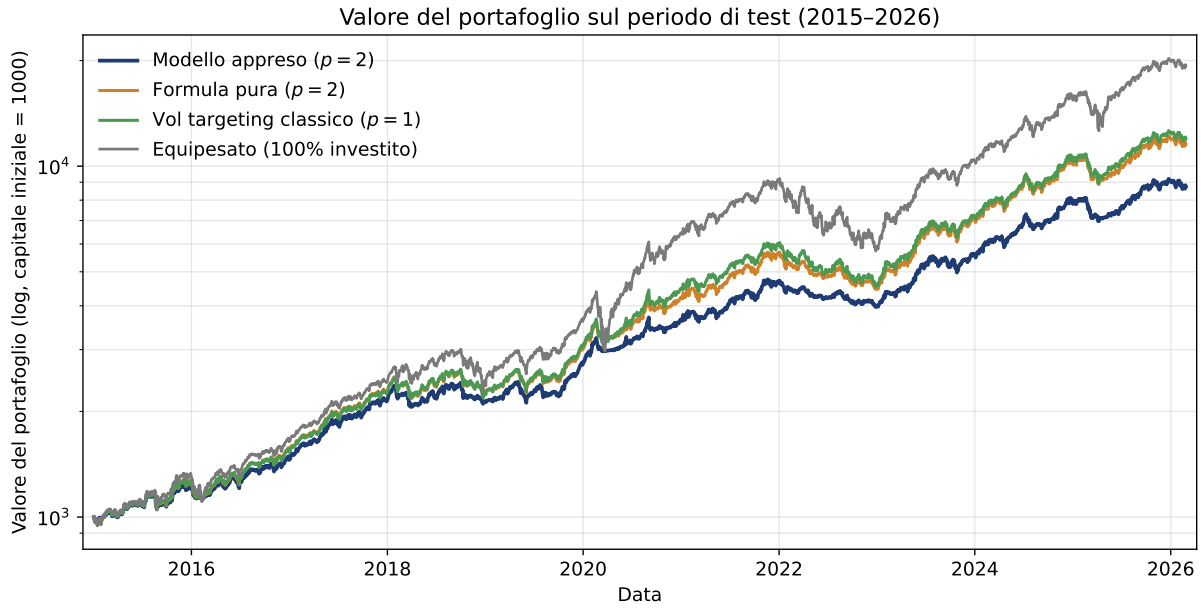


Figure 5.2: Valore del portafoglio nel tempo sul periodo di test (2015–2026): modello appreso, formula pura ($p = 2$), volatility targeting classico ($p = 1$) ed equipesato pienamente investito.

5.1.2 Esperimento 2: L'esponente al centro delle osservazioni

Dai risultati dello scorso esperimento abbiamo visto che il valore dell'esponente può fare la differenza in alcuni momenti del mercato grazie alla velocità e grandezza della reazione ai periodi di turbolenza, questa osservazione ci porta direttamente all'esperimento numero due. Quando l'esponente della formula è mantenuto fisso, il valore di p funge da principale leva difensiva, ma la sua efficacia dipende fortemente dal tipo di mercato:

- Nei crolli improvvisi e violenti (come il crash COVID del 2020 o il sell-off dei dazi del 2025) l'esponente $p = 1$ si comporta meglio. Questo perché riducendo l'esposizione in modo più moderato evita un'uscita eccessiva che farebbe perdere il rimbalzo successivo. Il vantaggio è evidente soprattutto sul rendimento corretto per il rischio (Sharpe e Calmar più alti), in quanto la strategia restando parzialmente investita partecipa alla ripresa del mercato.
- Nei ribassi lenti e prolungati (come il bear market del 2022) e sull'orizzonte storico di lungo periodo, un esponente $p = 2$ contiene meglio il drawdown. Quando la volatilità sale, anche in modo moderato ma persistente, la reazione amplificata dal quadrato riduce la quota investita in maniera più decisa, mantenendo più liquidità.

Nessun valore fisso, dunque, riesce a dominare su tutti i contesti di mercato.

Da questa constatazione abbiamo sviluppato due esperimenti paralleli:

- Rendere l'esponente p apprendibile dalla rete.

- Imporre una regola fissa che scambia il modello dalla versione con $p = 1$ a quella con $p = 2$ quando vengono superati determinati limiti.

Esponente appreso

Partendo dall'ipotesi che lasciar apprendere l'esponente alla rete neurale possa permettermi di adattarsi dinamicamente al contesto si è sviluppato il seguente esperimento.

Prima di tutto la rete è stata modificata aggiungendo una nuova `exposure_head` che si concentra sul calcolare l'esponente. Questa testa lavora in parallelo con quella progettata per l'esposizione. Per prendere la decisione utilizza gli stessi dati dell'altra che decide l'esposizione e ottimizza la stessa reward. La nuova testa può decidere valori dell'esponente compresi tra 1 e 2 e la sua scelta segue una funzione Sigmoidale.

Andiamo ora a osservare risultati dell'esperimento:

- **Sul fronte del rendimento corretto per il rischio (Sharpe ratio):** il modello con esponente appreso si dimostra inferiore. Le sue prestazioni restano sempre a metà strada tra $p = 1$ e $p = 2$.
- **Sul fronte della pura protezione dalle perdite (Drawdown):** a una prima lettura sembrerebbe un successo. Sul periodo 2015–2026, l'esponente appreso registra un MDD più contenuto e un Calmar ratio più elevato di entrambe le formule fisse. Questo potrebbe far pensare che l'agente abbia imparato a dosare la leva difensiva meglio delle regole statiche.

Ripetendo però lo stesso controllo di frontiera visto nell'Esperimento 1, questo presunto vantaggio svanisce. La minore perdita non nasce da una reale intelligenza di contesto, ma dal fatto che il modello si posiziona su un livello di liquidità media più alto di entrambe le regole a confronto.

A parità di capitale medio investito, le prestazioni del modello tornano a ricadere esattamente sulla frontiera deterministica (Fig. 5.1). Lo stesso identico livello di protezione è perfettamente replicabile abbassando il parametro σ_{target} della formula fissa.

In conclusione, anche sul drawdown, la metrica specifica per cui era stato ottimizzato, l'esponente appreso non aggiunge nulla che una configurazione strutturale leggermente più conservativa non offra già a costo zero. L'efficacia della protezione risiede interamente nelle scelte a monte dell'architettura, non nei pesi dinamici appresi dalla rete.

Table 5.3: Esperimento 2: esponente p dell'overlay a confronto (DD_VOL_PENALTY, $\sigma_{\text{target}} = 0.20$, mediana su 3 seed). $p = 1$ vol targeting, $p = 2$ variance targeting, p appreso regime-dipendente.

Periodo	Esponente	cash	MDD	Calmar	Sharpe
2015–26	$p = 1$	0.155	0.251	1.00	1.430
	$p = 1.5$	0.239	0.191	1.16	1.456
	$p = 2$	0.220	0.184	1.24	1.463
	appreso	0.229	0.164	1.39	1.447
COVID	$p = 1$	0.431	0.153	3.41	2.280
	$p = 1.5$	0.584	0.102	3.76	2.438
	$p = 2$	0.579	0.097	4.18	2.477
	appreso	0.612	0.085	4.61	2.470
2022	$p = 1$	0.476	0.252	-0.95	-1.417
	$p = 1.5$	0.650	0.187	-0.95	-1.520
	$p = 2$	0.688	0.181	-0.95	-1.616
	appreso	0.757	0.162	-0.96	-1.736
2017	$p = 1$	0.005	0.047	9.67	3.132
	$p = 1.5$	0.007	0.047	9.68	3.122
	$p = 2$	0.005	0.047	9.67	3.132
	appreso	0.005	0.047	9.67	3.132
2025	$p = 1$	0.129	0.174	0.99	1.038
	$p = 1.5$	0.191	0.148	0.99	1.022
	$p = 2$	0.173	0.152	0.95	0.984
	appreso	0.178	0.149	0.96	0.932

Una loss dedicata all'esponente Dall'esperimento di sopra abbiamo dimostrato che utilizzando la stessa reward dell'esposizione non riusciamo a battere un esponente fisso. Non essendo la reward specificatamente progettata per l'esponente ci si è chiesto se implementare una loss dedicata potrebbe portare a dei risultati vincenti. In questo esperimento si è provato ad utilizzare una loss progettata specificatamente per addestrare la testa che decide l'esponente a prevedere quando la volatilità futura darà vita a un crollo. In sostanza se la testa prevede un crollo imposta l'esponente $p = 2$ altrimenti a $p = 1$.

Per questo specifico problema si parla di apprendimento supervisionato. È stata costruita un'etichetta che vale 1 quando il mercato scende di oltre il 10% nei 21 giorni successivi, ovvero avviene un crollo, e 0 altrimenti. La testa produce una probabilità di crollo prevista $\hat{P}(\text{crollo}) = \sigma(z) \in (0, 1)$. La loss usata per questo esperimento è una cross-entropy binaria (BCE). Ad ogni step la funzione di loss misura quanto la previsione del modello è distante dalla verità. Penalizza di più in relazione a quanto il modello è

sicuro di una previsione errata, cercando di spingere il modello a fare la previsione giusta. La probabilità di crollo prevista infine viene sommata a uno per ricavare l'esponente finale $p = 1 + \hat{P}(\text{crollo})$.

Ancora prima di guardare i risultati dei backtest è venuto alla luce un altro problema: la loss va in stallo (Fig. 5.3). All'inizio dell'addestramento la loss sembrava funzionare e l'errore scendeva gradualmente in modo corretto. Arrivati circa a metà la loss si è fermata ad un valore alto, vicino a quello di un predittore privo di informazione. Dall'inizio dello stallo in poi la loss ha continuato ad oscillare sempre attorno allo stesso valore, a causa del rumore dei vari mini-batch, e scendendo di un valore irrisorio fino alla fine dell'addestramento. Questo è avvenuto perché il modello ha raggiunto un tetto informativo, cioè partendo dai dati non gli è stato possibile estrarre l'informazione sulla direzione del mercato. Senza informazione direzionale non gli è stato possibile distinguere un crollo in arrivo da un semplice ribasso lento del mercato e questo lo ha portato nello stallo.

Dai backtest (Tab. 5.4) si mostra come la variante supervisionata non batte né l'esponente appreso dal solo reward né la formula fissa $p = 2$: presenta anzi un MDD più alto in ogni periodo, tenendo meno cash, senza guadagno su Calmar o Sharpe.

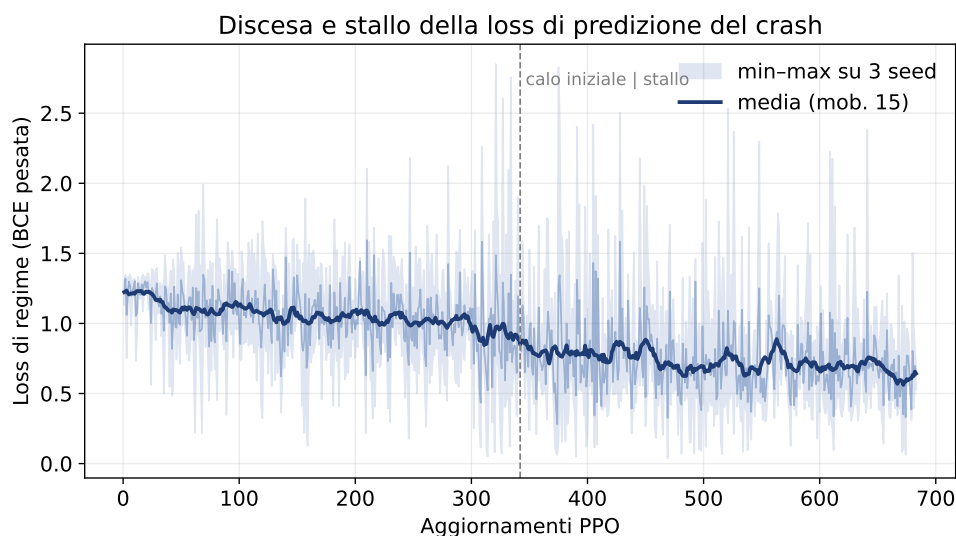


Figure 5.3: Loss BCE di predizione del crash durante l'addestramento (media su 3 seed, banda min-max).

Table 5.4: Esponente con loss di regime (BCE) vs esponente appreso dal solo reward (Esp. 2) vs formula fissa $p = 2$, universo large-cap (mediana su 3 seed).

Periodo	Strategia	cash	MDD	Calmar	Sharpe
2015–26	formula $p=2$	0.34	0.138	1.35	1.44
	appreso (reward)	0.37	0.122	1.41	1.42
	appreso + BCE	0.31	0.166	1.20	1.44
COVID	formula $p=2$	0.71	0.067	4.13	2.52
	appreso (reward)	0.74	0.059	4.06	2.50
	appreso + BCE	0.63	0.090	3.49	2.32
2022	formula $p=2$	0.78	0.128	−0.95	−1.60
	appreso (reward)	0.81	0.111	−0.95	−1.61
	appreso + BCE	0.68	0.155	−0.95	−1.50
2025	formula $p=2$	0.27	0.119	1.16	1.09
	appreso (reward)	0.31	0.111	1.19	1.10
	appreso + BCE	0.27	0.123	1.09	1.09

In sintesi, fornire una supervisione esplicita all'esponente non cambia l'esito. Il limite non è la modalità di addestramento ma l'assenza nei dati di un segnale sfruttabile sulla direzione futura dei rendimenti.

Regolare l'esponente con una regola

Dato le precedenti osservazioni abbiamo pensato di affidare la scelta su quale esponente usare a una regola fissa. Questo permette di adattare dinamicamente l'esponente in base alla volatilità di mercato. La regola implementata consiste nell'aumentare l'aggressività del taglio di esposizione, cioè l'esponente p , man mano che sale l'indice VIX.

Tuttavia, anche questa soluzione fallisce: la regola ibrida riesce a eguagliare o superare la miglior formula fissa in **al più uno scenario su cinque**, sia in termini di drawdown che di Sharpe ratio.

Ci sono tre ragioni principali per cui questa soluzione non funziona:

1. **Il compromesso diluisce l'efficacia:** variare p in modo progressivo crea un'esposizione intermedia che finisce per non essere ottimale in nessun contesto: taglia troppo poco nei crolli improvvisi e troppo presto nelle fasi di lenta correzione.
2. **Il VIX è un segnale reattivo, non predittivo:** il VIX non anticipa le crisi, ma esplode quando il crollo è già iniziato. Quando la regola rileva un picco di volatilità e decide finalmente di alzare l'esponente a $p = 2$, la perdita di capitale si è già avvenuta in gran parte.

3. **Asimmetria informativa:** capire se una discesa dei mercati sia l'inizio di un ribasso prolungato (dove servirebbe $p = 1$) o di uno shock istantaneo (dove servirebbe $p = 2$) è possibile solo a posteriori, mai in tempo reale mentre il fenomeno sta avvenendo.

Quindi, un sistema difensivo basato su un segnale che arriva tardi non può impedire il danno. È come chiudere la porta dopo che il ladro è entrato. Il meccanismo di difesa si attiva, ma ormai i danni sono fatti.

Table 5.5: Esperimento 3: regola morbida sul VIX ($p : 1 \rightarrow 2$ via sigmoide) contro $p = 1$ e $p = 2$ fissi, in inferenza dallo stesso checkpoint ($\sigma_{\text{target}} = 0.20$, mediana su 3 seed).

Periodo	Modalita'	cash	MDD	Calmar	Sharpe
2015–26	$p = 1$ fisso	0.171	0.240	1.02	1.433
	$p = 2$ fisso	0.220	0.184	1.24	1.463
	regola VIX	0.183	0.234	1.01	1.432
COVID	$p = 1$ fisso	0.451	0.148	3.41	2.281
	$p = 2$ fisso	0.579	0.097	4.18	2.477
	regola VIX	0.510	0.122	3.62	2.380
2022	$p = 1$ fisso	0.504	0.240	−0.95	−1.421
	$p = 2$ fisso	0.688	0.181	−0.95	−1.616
	regola VIX	0.550	0.235	−0.95	−1.534
2017	$p = 1$ fisso	0.005	0.047	9.67	3.129
	$p = 2$ fisso	0.005	0.047	9.67	3.132
	regola VIX	0.005	0.047	9.67	3.130
2025	$p = 1$ fisso	0.140	0.171	1.00	1.048
	$p = 2$ fisso	0.173	0.152	0.95	0.984
	regola VIX	0.149	0.165	0.90	0.982

5.1.3 Generalità: trasferimento su panieri diversi

Per verificare se im modello abbia davvero la proprietà di generalità o se sia invece legato al paniere azionario su cui è stata addestrato, abbiamo fatto un test di trasferimento. Abbiamo applicato il modello direttamente a portafogli completamente nuovi, senza fargli vedere prima i dati e senza alcun ri-addestramento.

Grazie alla sua architettura, il modello può gestire portafogli di qualsiasi dimensione e composizione. Lo abbiamo quindi messo alla prova su panieri dell'indice S&P 500 molto eterogenei per volatilità, settore di appartenenza, numerosità ($N = 5$ e $N = 20$) e grado di correlazione interna.

Sul decennio 2015–2026 (Tab. 5.6), la capacità difensiva del modello dimostra di saper **generalizzare con successo**:

- **Difesa attiva reale** ($MDD_gain > 0$): su tutti i sei panieri testati, il modello contiene la perdita massima meglio di una strategia passiva con identica liquidità media (cioè il MDD del modello è minore rispetto a una strategia che tiene una liquidità sempre pari alla media del modello dal 2015 al 2026). Si tratta quindi di una gestione dinamica che aggiunge valore concreto.
- **Reattività anticiclica** ($\rho_{cash,dd} > 0$): la correlazione positiva conferma che il modello alza la quota di contante ogni volta che il mercato comincia a crollare, a prescindere dal tipo di titoli in portafoglio.
- **Flessibilità sul numero di titoli**: il meccanismo funziona altrettanto bene sia con un paniere ristretto di sole 5 azioni (*small5*) sia con uno più ampio di 20 (*large20*). Inoltre, nel paniere ben diversificato e a bassa correlazione le prestazioni restano solide, eliminando la paura che mantenere fissi i titoli nel tempo penalizzasse la strategia.

Dai risultati emerge un vincolo strutturale: la difesa è simmetrica ($down_capt \approx up_capt$). Il modello reagisce puramente alla volatilità complessiva del paniere a prescindere dal fatto che sia a rialzo o ribasso. Non c'è quindi alcun tipo di previsione direzionale.

Table 5.6: **Trasferimento sul periodo lungo (2015–2026)**. Modello finale ($p = 2$, mediana su 3 seed) su panieri mai visti in addestramento. $MDD_gain > 0$: drawdown ridotto meglio di un cash fisso di pari entità media; $\rho_{cash,dd}$: tendenza ad alzare il contante al peggiorare delle perdite.

Paniere	MDD	Calmar	MDD_gain	down/up capt	$\rho_{cash,dd}$
difensivi	0.150	0.68	+0.115	0.92/0.92	0.38
diversificato	0.156	0.84	+0.165	0.87/0.86	0.69
large20	0.241	0.85	+0.059	0.82/0.82	0.65
small5	0.169	0.81	+0.165	0.87/0.86	0.71
ciclici	0.383	0.16	+0.109	0.71/0.68	0.65
growth	0.201	0.95	+0.150	0.52/0.52	0.59

Il riscontro più significativo si vede dallo stress test nei regimi critici (Tab. 5.7): insieme alla capacità difensiva, si trasferisce in modo identico anche il limite dell'approccio.

- **Nei crolli violenti e rapidi (Crash COVID 2020)**: la volatilità esplode immediatamente facendo scattare subito i tagli di esposizione. Il modello ottiene una vittoria netta su ogni paniere ($MDD_gain > 0$ ovunque).
- **Nei mercati ribassisti lenti e logoranti (Bear market 2022)**: il modello fallisce su tutti i panieri ($MDD_gain \leq 0$). In questo scenario i prezzi scendono

con costanza ma senza picchi clamorosi di volatilità. Il modello quindi non riceve quei segnali che gli indicano quando ritirarsi dal mercato in maniera aggressiva. Per questo tiene sempre la liquidità abbastanza alta ma non ha nessun timing utile da sfruttare finendo per far peggio di un null a cash costante.

Questo schema è esattamente lo stesso osservato sui dati di addestramento. Il fatto che compaia anche su panieri del tutto nuovi ne dimostra la natura universale: non è un'anomalia di un singolo set di azioni, ma una proprietà strutturale del volatility targeting.

Table 5.7: Difesa nei regimi di stress, zero-shot (modello finale $p = 2$, mediana su 3 seed).

Paniere	MDD_gain (COVID)	MDD_gain (2022)
difensivi	+0.096	−0.003
diversificato	+0.056	−0.011
growth	+0.041	−0.027
large20	+0.065	−0.034
small5	+0.057	−0.007
ciclici	+0.002	−0.013

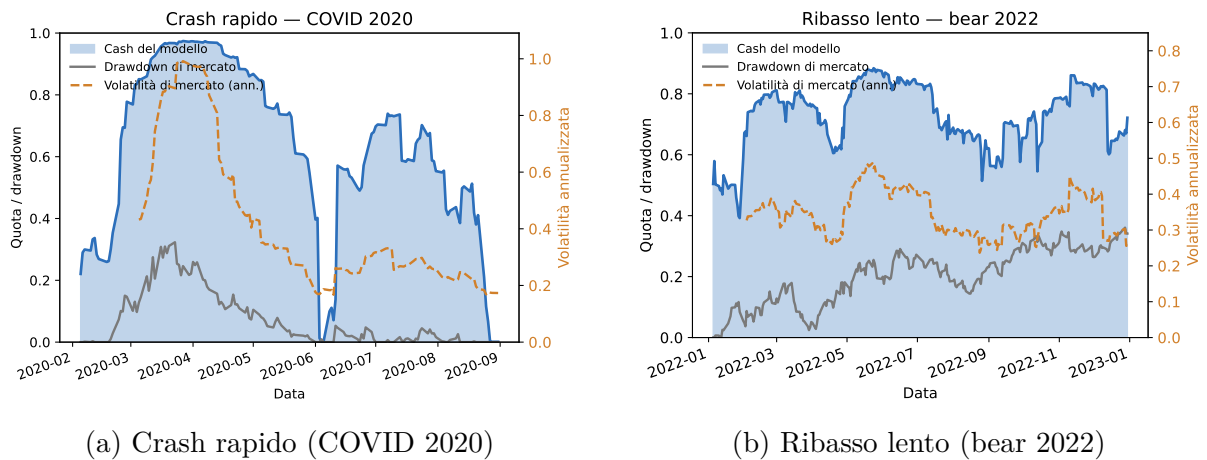


Figure 5.4: Allocazione nel tempo: quota di liquidità del modello sovrapposta al drawdown/volatilità di mercato. Nel crash rapido (COVID) il cash sale in tempo; nel bear lento (2022) la volatilità resta bassa e la difesa arriva tardi.

5.1.4 Filo conduttore

Ogni leva analizzata: m_θ calcolato, esponente stimato, regola in regime, non fornisce nulla che non si possa ottenere già scegliendo i parametri strutturali fissi (σ_{target} e esponente). Il valore è completamente nella struttura, non nell'adattamento. La dipendenza dal regime è una caratteristica del mercato e non può essere utilizzata in tempo reale perché il regime

è noto solo a posteriori. Il modello e la sua difesa si adattano a diversi universi, compresa la debolezza strutturale nei mercati lenti.

Chapter 6

Applicazione Mobile

Oltre al modello è stata implementata anche un'applicazione mobile per android.

6.1 Obiettivo dell'applicazione

L'obiettivo principale dell'applicazione è rendere fruibile iò che la parte di ricerca analizza soltanto in backtest. Il modello DRL, le strategia del volatility targeting e dell'equipesato vengono portate fuori dall'ambiente sperimentale assieme all'inverse volatility e al principio di Kelly. Queste ultime due sono state aggiunte per dare all'utente la possibilità di osservare come si comportano strategie cn obiettivi diversi, il kelly in particolare non si concentra sul minimizzare la perdita ma massimizzare il guadagno. L'utente può applicare queste strategie a un proprio paniere di titoli e ciò gli permette di fare lui gli esperimenti che vuole. In questo modo i metodi analizzati diventano uno strumento realmente utilizzabile.

A questo obiettivo principale se ne affiancano due secondari.

Confronto ed esplorazione delle strategie. L'applicazione permette all'utente di osservare, sullo stesso paniere, i pesi che ciascuna strategia assegnerebbe e di valutarle tramite un backtest comparativo con le metriche principali (Sharpe, maximum drawdown, Calmar). Diventa così uno strumento per toccare con mano il risultato centrale di questo lavoro: il modello appreso e la formula fissa producono allocazioni molto simili, rendendo evidente anche all'utente la differenza tra una difesa ottenuta da una reale capacità predittiva e una dovuta alla semplice conservatività.

Position sizing accessibile. Riprendendo la motivazione dell'introduzione, ovvero i bias comportamentali che compromettono il dimensionamento della posizione fatto a intuito, l'applicazione offre uno strumento di position sizing sistematico e consapevole del rischio, utilizzabile anche da chi non possiede una conoscenza approfondita della teoria finanziaria.

6.2 Interfaccia e funzionalità

In questa sezione si descrive l'interfaccia dell'applicazione, organizzata attorno a due schermate principali. La UI è pensata per essere semplice e intuitiva: un utente deve poter capire facilmente come muoversi all'interno dell'app e sfruttare le varie funzionalità. Non è richiesta all'utente una conoscenza approfondita della teoria finanziaria per poter usufruire dell'app, anche se ovviamente è saggia almeno come funzionano le varie strategie a grandi linee.

6.2.1 Dashboard e gestione del portafoglio

In alto un selettore permette di passare tra i vari portafogli. La carta centrale mostra il valore corrente del portafoglio e il guadagno o la perdita rispetto al capitale iniziale. In alto a destra si trova un'icona che permette di creare nuovi portafogli. Cliccando su di essa si aprirà un form che chiederà all'utente di inserire obbligatoriamente nome e capitale iniziale del portafoglio, e opzionalmente costo delle commissioni in bps (unità di misura che equivale alla centesima parte di un punto percentuale) e costo base di acquisto, se non si inseriscono questi ultimi due valori saranno impostati di default a zero. Appena sotto la carta sono presenti i pulsanti per eliminare il portafoglio o riavviarlo al capitale di partenza e azzerando le allocazioni.

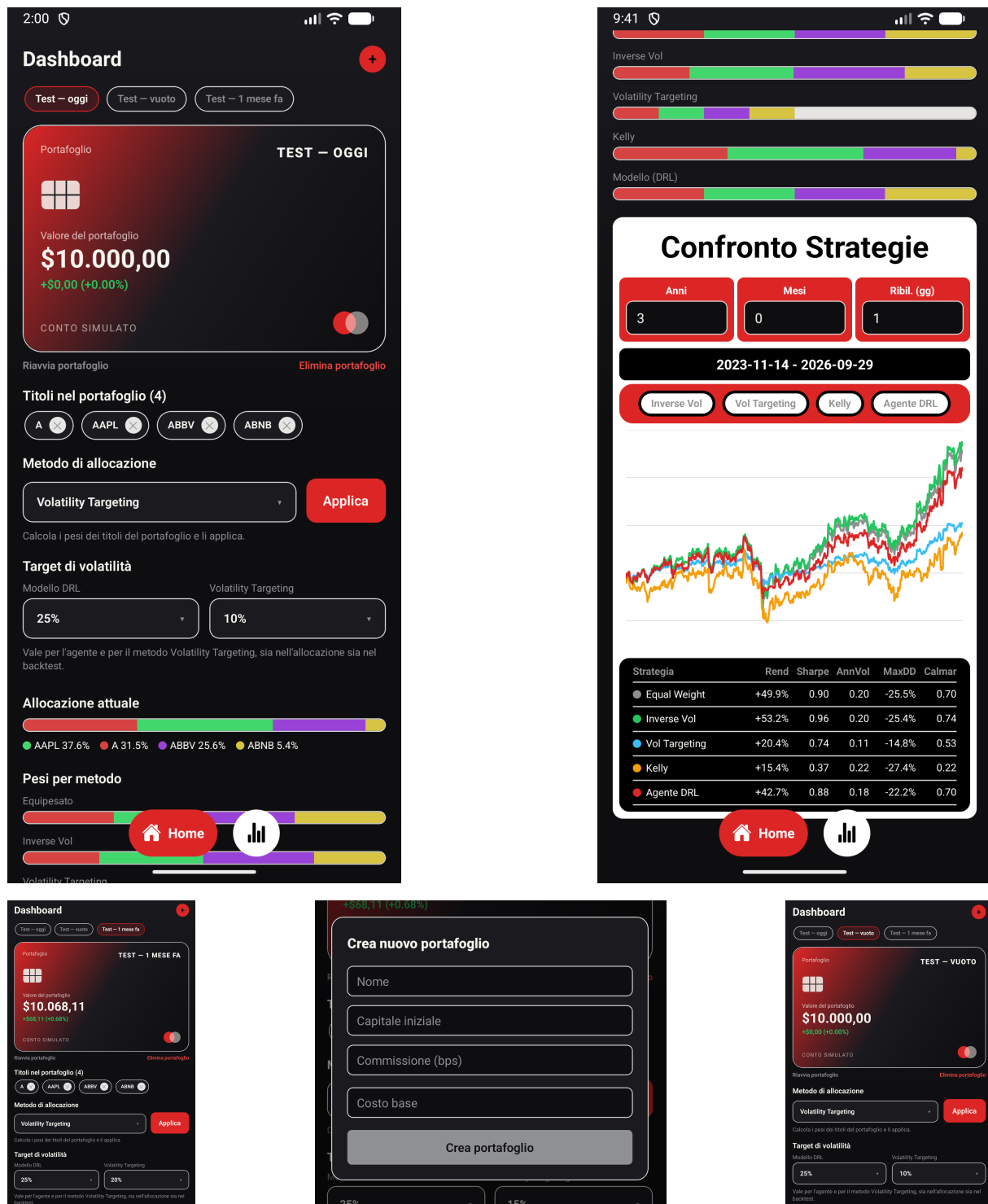


Figure 6.1: Dashboard: le carte dei tre portafogli simulati (oggi, un mese fa, vuoto) e la finestra di creazione di un nuovo portafoglio.

Sotto la carta sono elencati i titoli del portafoglio con la possibilità di rimuoverli. Se si sceglie di rimuovere un titolo la sua quota torna in liquidità; i pesi degli altri titoli restano invariati se si vuole aggiornarli bisogna riapplicare una strategia tra quelle proposte. Il guadagno/perdita accumulato da quel titolo viene scartato, quindi il valore del portafoglio si adegua di conseguenza.. Invece i dati nella sezione sul backtest e i pesi dati dalle varie

strategie si riaggiornano subito appena il titolo viene eliminato dal portafoglio. Sotto questa sezione con i titoli del portafoglio ci sono le sezioni per scegliere quale strategia di allocazione applicare al portafoglio. L'utente può applicare al portafoglio una tra cinque strategie di allocazione: Equipesato, Inverse Volatility, Volatility Targeting, Kelly e il modello DRL, queste vengono descritte in dettaglio nella sezione 6.4. Per il modello DRL e il Volatility Targeting può inoltre scegliere il target di volatilità tramite appositi selettori. Le opzioni per questi ultimi due sono i target più comuni usati in finanza. Applicando una strategia, i pesi vengono calcolati e assegnati ai titoli, con l'addebito dei relativi costi di transazione.

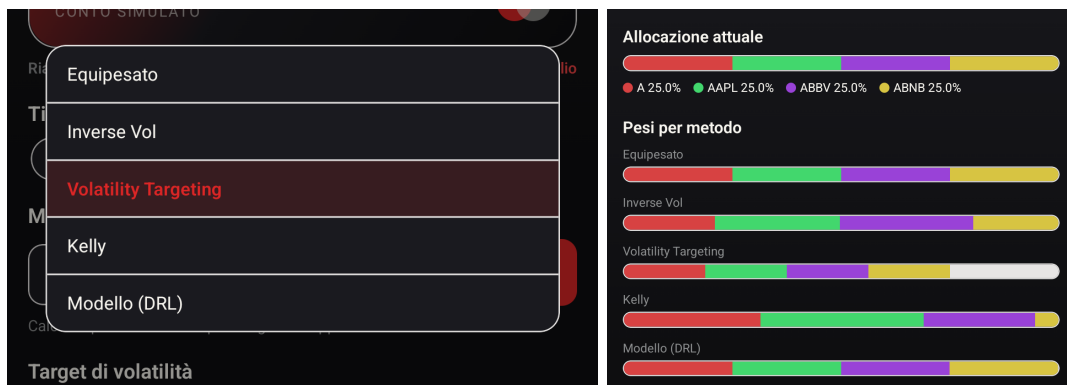


Figure 6.2: Selezione del metodo di allocazione e visualizzazione dei pesi risultanti per ciascun metodo.

Infine, un grafico di confronto mostra l'andamento simulato backtest del capitale secondo le diverse strategie, affiancato da una tabella con le metriche principali. È possibile inoltre scegliere che periodo deve coprire il backtest in anni e mesi (massimo 3 anni) e ogni quanti giorni deve avvenire il ribilanciamento in esso. In questo modo l'utente può vedere come sarebbero andate queste varie strategie nel tempo e grazie all'opzione del ribilanciamento può anche sperimentare in minima parte delle strategie sue.

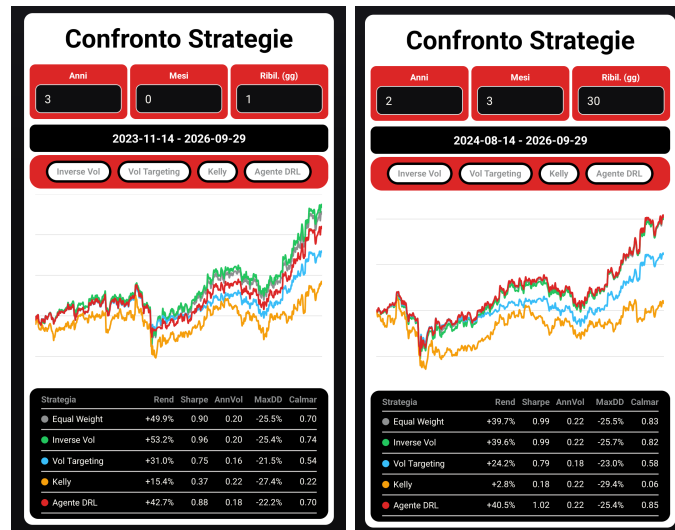


Figure 6.3: Confronto delle strategie tramite backtest, con tabella delle metriche.

6.2.2 Elenco dei titoli

La seconda schermata mostra l'elenco dei titoli dell'indice S&P 500. Ogni voce riporta il logo dell'azienda, il prezzo più recente, un mini-grafico (sparkline) dell'andamento e la variazione percentuale, colorata in verde o rosso. Una barra di ricerca in alto permette di filtrare i titoli per ticker.

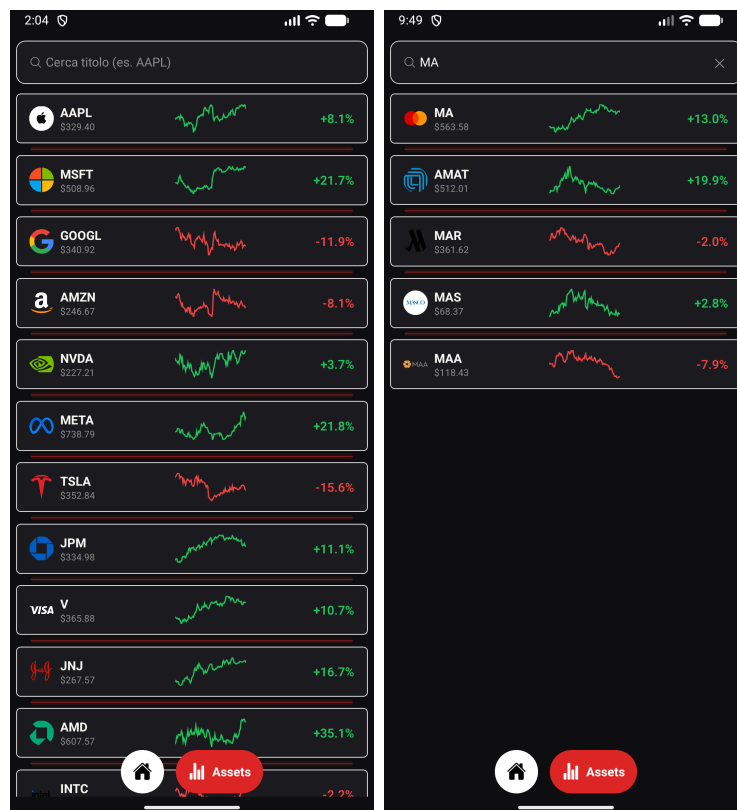


Figure 6.4: Elenco dei titoli con barra di ricerca e card riepilogative.

6.2.3 Dettaglio del titolo

Toccando un titolo si apre la schermata di dettaglio, con il prezzo, la variazione e un grafico che può essere visualizzato come candele o come linea, su orizzonti temporali selezionabili (una settimana, un mese, un anno, oppure l'intero storico). Da qui l'utente può aggiungere il titolo a uno dei propri portafogli tramite un apposito selettore.

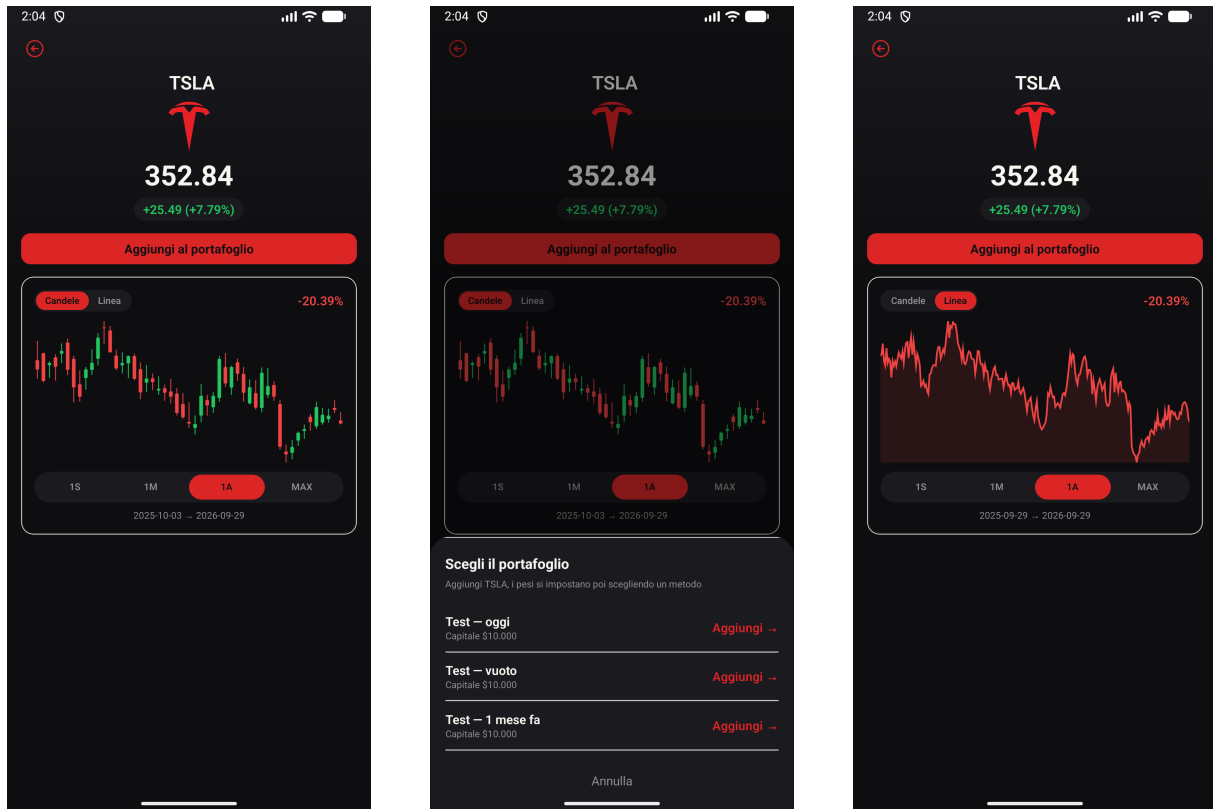


Figure 6.5: Dettaglio del titolo: grafico a candele (sinistra) e a linea (destra), e, al centro, la selezione del portafoglio a cui aggiungere il titolo.

6.3 Dettagli implementativi

In questa sezione si espongono le scelte implementative più interessanti adottate per lo sviluppo dell'applicazione.

L'applicazione adotta un'architettura local-first cioè tutta l'elaborazione dei dati e le funzioni principali dell'app come il calcolo dei pesi e l'inferenza del modello avvengono sul dispositivo, l'unica interazione con la rete è il download periodico dei prezzi. I motivi dietro questa scelta sono:

- **Privacy e autonomia:** i portafogli dell'utente, le allocazioni e i risultati non lasciano mai il dispositivo.

- **Funzionamento offline:** dopo la prima sincronizzazione l'app resta completamente utilizzabile anche senza connessione in quanto tutti i dati utili al suo funzionamento risiedono in un database locale.
- **Costo nullo di infrastruttura:** non essendoci un backend da mantenere, l'aggiornamento dei dati è delegato a un servizio di automazione gratuito, evitando qualsiasi hosting dedicato.

In sostanza c'è una separazione tra due sottosistemi: una pipeline di produzione del dato e un sottosistema di consumo e persistenza.

Sorgente e struttura del dato

I dati di mercato sono ottenuti da Yahoo Finance tramite la libreria `yfinance`. L'universo scaricato è l'intero indice S&P500: l'elenco dei ticker è estratto a runtime dalla pagina di Wikipedia dell'indice, in modo da garantire l'aggiornamento automatico della composizione qualora dovesse cambiare nel tempo, nel caso in cui Wikipedia non dovesse essere raggiungibile viene usata una lista statica dei titoli nell'S&P500. Per ogni titolo viene acquisita una storia di tre anni di dati giornalieri, con la modalità `auto_adjust = True` come per il modello. Le feature che vengono conservate sono le quattro feature di mercato utilizzate dal modello cioè `high`, `low`, `close` e `VIX`, l'`open` non viene memorizzata in quanto non viene utilizzata da alcun metodo. Il download avviene un ticker alla volta e non in blocco, in questo modo vengono prodotte serie a indice singolo più semplici e robuste da elaborare, e permette di isolare i fallimenti.

Pipeline di aggiornamento automatico

L'aggiornamento è affidato a un workflow di GitHub Actions, l'esecuzione avviene alle 22:00 UTC nei giorni feriali, ovvero dopo la chiusura del mercato statunitense. Il workflow:

1. predispone un ambiente Python ed installa le dipendenze;
2. esegue lo script di download, che produce il dataset;
3. effettua il commit del dataset aggiornato nel repository.

Il repository Git assume il ruolo di archivio versionato dei dati e punto di distribuzione verso i client. Con tutti gli storici dei 503 titoli però esso raggiunge dimensioni di circa 33MB che è una dimensione molto grande per essere scaricata sui dispositivi mobili.

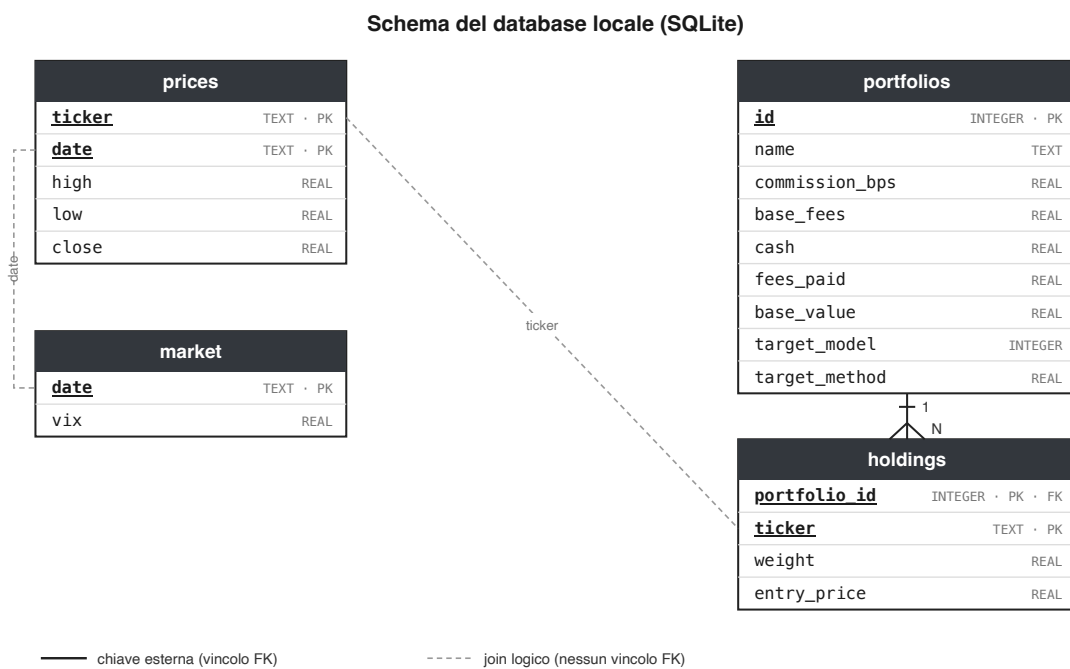
6.3.1 Datasets

Il Dataset è in formato JSON, con due collezioni principali: `prices` e `market`. Vista la dimensione elevata, viene applicata una conversione gzip al momento della generazione del

dataset: lo script genera accanto al JSON una versione compressa che pesa circa 5,4MB. La versione gzip è quella effettivamente usata, lato client la decompressione avviene attraverso la libreria `fflate`, in javascript puro.

Database locale

Sul dispositivo i dati vengono racchiusi in un database SQLite, interrogato tramite l'ORM tipizzato Drizzle. Lo schema è minimale e normalizzato, ed è diviso nelle seguenti tabelle:



Le chiavi primarie composte (`ticker, date`) e (`portfolio_id, ticker`) impediscono che ci siano dati duplicati. L'unica relazione con vincolo di integrità referenziale è `holdings.portfolio_id → portfolios.id`.

SQLite su dispositivi mobili ha un limite di variabili che può gestire con un'unica query. Per risolvere questo problema i dati vengono divisi in array da al massimo 500 elementi ciascuno. I dati vengono quindi inseriti un blocco alla volta all'interno del database in modo da evitare errori.

L'evoluzione dello schema è gestita tramite migrazioni versionate, generate in fase di sviluppo e applicate automaticamente all'avvio prima di ogni accesso ai dati. Un registro interno assicura che ogni migrazione sia applicata una sola volta, così se il database è già allineato l'avvio non esegue alcuna istruzione.

Sincronizzazione lato applicazione

All'avvio, dopo le migrazioni, l'app esegue la procedura di sincronizzazione, che svolge tre operazioni fondamentali.

- **Limitazione di frequenza:** una tabella di servizio conserva la data dell'ultima sincronizzazione e se coincide con quella odierna impedisce che vengano fatte altre sincronizzazioni.
- **Inserimento idempotente:** come descritto nella sezione precedente i dati vengono inseriti a blocchi. In caso di conflitto sulle chiavi primarie la riga esistente viene aggiornata con i valori piu' recenti. In questo modo si evitano duplicazioni di dati.
- **Finestra temporale mobile:** ad ogni sincronizzazione vengono scaricati gli ultimi tre anni di storico. Si scaricano gli interi tre anni perché `yfinance` con `auto_adjust=True` aggiorna i prezzi compresi quelli passati ad ogni split e dividendo, quindi vanno sovrascritti. Per tenere lo spazio occupato costante vengono eliminati di volta in volta tutti i record corrispondenti a date precedenti alla prima data nel dataset scaricato.

Inizializzazione e primo avvio

L'applicazione include un seed iniziale di dati impacchettati nel bundle per garantire l'usabilità anche in assenza di rete alla prima apertura. Al primissimo avvio, se la tabella dei prezzi è vuota, il seed la popola. Successivamente la sincronizzazione allinea il contenuto alla versione più recente.

Limiti

Si espongono in questa sezione limiti progettuali dell'app. I dati vengono scaricati solo una volta al giorno e sono i prezzi a chiusura del mercato, non ci sono quindi dati in tempo reale. L'API di `yfinance` si appoggia a end-point non ufficiale di Yahoo Finance, il servizio può cambiare senza preavviso e richiederebbe manutenzione dello script. La distribuzione via repository nn è pensata come canale ad alto volume, funziona bene per essere usata da pochi utenti ma se diventano migliaia non reggerebbe la concorrenza.

6.3.2 Esecuzione del modello on-device

Il modello discusso nella sezione 4.1 viene addestrato in Python ed esportato in formato ONNX (Open Neural Network Exchange). ONNX è uno standard e serve a disaccoppiare il framework di addestramento dal motore di esecuzione. In questo modo è possibile far girare il modello in un ambiente React Native direttamente sul dispositivo.

L'inferenza quindi avviene in locale utilizzando la libreria `onnxruntime-react-native` che esegue il modello sulla CPU del dispositivo. Non è previsto alcun server di inferenza, in linea con l'architettura local-first dell'app.

Sono stati addestrati tre modelli con target di volatilità 15,20 e 25% e i loro file ONNX sono stati inclusi come assets nel bundle dell'applicazione. La scelta del target è salvata per singolo portafoglio e decide quale dei tre modelli viene eseguito, vale lo stesso per il target del metodo Target Volatility. La sessione di inferenza per ogni modello viene creata una sola volta, al primo utilizzo, e viene conservata in un singleton di modulo e riutilizzata per tutte le previsioni successive. Il costo di caricamento del modello si paga quindi una volta sola, in background, senza bloccare l'interfaccia. Un meccanismo di guardia impedisce inizializzazioni concorrenti.

Ogni previsione segue tre passi:

1. si costruiscono i tre tensori di input: `features`, `prev_weights` e `risk_state`, in formato `float32`;
2. si esegue la sessione con `session.run(feeds)`;
3. si legge il tensore di output `portfolio_weights`, i pesi del portafoglio che sommano a 1.

Questa procedura è usata in due funzionalità diverse. Viene usata per l'allocazione in tempo reale quando l'utente applica il metodo al suo portafoglio facendo partire una previsione usando i dati più recenti. E viene usata anche per il backtest in cui il modello viene interrogato ad ogni ribilanciamento.

6.3.3 Motore di backtest

È stato implementato un motore di backtest walk-forward per simulare e confrontare fra loro le strategie di allocazione proposte nell'app. Questo motore simula le decisioni che avrebbero fatto giorno per giorno le varie strategie utilizzando solo i dati passati. Quindi tutte le strategie ricevono un contesto di soli dati storici e restituiscono i pesi target, in questo modo si garantisce un confronto equo in quanto i risultati dipendono solo dai pesi prodotti.

Tutte le serie di dati vengono prima allineate sulle date comuni, così che ogni strategia lavori sullo stesso insieme di giorni. La simulazione procede poi con un capitale iniziale normalizzato, dove i pesi rappresentano le frazioni investite e la quota rimasta resta in liquidità. Ad ogni passo:

1. **Ribilanciamento** ogni N giorni di borsa: alla strategia viene passata una finestra di soli dati passati insieme allo stato corrente del portafoglio. La strategia restituisce i pesi target.

2. **Costi di transazione:** si calcola il turnover, cioè la somma delle variazioni di peso rispetto all'allocazione precedente, e si addebita i costi di transazione derivanti. In questo modo strategie che ribilanciano in modo più aggressivo vengono penalizzate realisticamente.
3. **Evoluzione giornaliera:** si applica al capitale il rendimento del portafoglio del giorno, $r_p = \sum_i w_i r_i$. Questo rendimento non è visibile alla strategia ma viene calcolato dopo la decisione, garantendo l'assenza di look-ahead.
4. **Drift dei pesi:** tra un ribilanciamento e l'altro i pesi non restano fissi ma evolvono come in un buy-and-hold, i titoli che crescono di più aumentano il proprio peso relativo secondo $w_i \leftarrow w_i(1 + r_i)/(1 + r_p)$.

Al termine, dal periodo di backtest si calcolano le metriche di valutazione: rendimento totale, rendimento annualizzato, volatilità annualizzata, Sharpe ratio, maximum drawdown e Calmar Ratio.

6.3.4 Validazione numerica

Dato che l'intera logica quantitativa è stata re-implementata in TypeScript per girare sul dispositivo, è stato necessario verificare che i risultati coincidano con quelli dell'ambiente di riferimento in Python. Si sono quindi messi a confronto gli output delle due implementazioni sugli stessi input.

Le funzioni finanziarie e il motore di backtest sono stati confrontati con un'implementazione di riferimento basata su NumPy. Gli scostamenti rilevati sono dell'ordine 10^{-16} . Le due implementazioni producono di fatto gli stessi risultati non contando gli errori di arrotondamento inevitabili.

Per l'agente si è confrontata l'inferenza on-device tramite ONNX Runtime con i pesi prodotti dal modello di riferimento in Python, sugli stessi dati storici. Lo scostamento mediano per singolo asset è risultato di circa 0,3 punti percentuali. Questa piccola differenza è dovuta alla precisione ridotta data dai dati in float32 sul dispositivo contro float64 in Python. Anche qui però la differenza è minima e trascurabile e di fatto l'esecuzione on-device replica fedelmente quella del modello addestrato.

Si è dimostrato quindi che le decisioni prese dall'app sul dispositivo sono equivalenti a quelle calcolabili nell'ambiente di sviluppo.

6.4 Strategie

In questa sezione viene mostrata l'implementazione delle varie strategie all'interno dell'applicazione.

Il modello e la sua implementazione nell'app sono già stati descritti nei capitoli e sezioni precedenti.

6.4.1 Equipesato

L'equipesato consiste semplicemente nel dividere il capitale equamente tra i vari titoli nel portafoglio.

6.4.2 Inverse Volatility

L'idea alla base di questa strategia consiste nell'assegnare il capitale in modo inversamente proporzionale alla volatilità di ciascun asset, penalizzando i titoli più nervosi e premiando quelli più stabili.

Il calcolo dell'allocazione si divide in tre passaggi principali:

1. **Misura del rischio:** Per ogni asset i , si calcola la volatilità storica σ_i applicando la deviazione standard campionaria sui rendimenti.
2. **Inversione:** Si calcola il reciproco della volatilità $\frac{1}{\sigma_i}$. Se un titolo presenta un'elevata volatilità, il suo inverso risulterà basso, al contrario per un titolo stabile il reciproco assume un valore più elevato.
3. **Normalizzazione:** Si divide l'inverso della volatilità del singolo asset per la somma degli inversi di tutti gli asset del portafoglio. La somma finale dei pesi w_i è pari a 1.

La formula matematica per il calcolo del peso w_i dell'asset i -esimo è definita come:

$$w_i = \frac{\frac{1}{\sigma_i}}{\sum_{j=1}^N \frac{1}{\sigma_j}} \quad (6.1)$$

dove w_i è il peso dell'asset i , σ_i la deviazione standard campionaria dei suoi rendimenti e N il numero di asset.

6.4.3 Kelly Criterion

Il criterio di Kelly calcola l'esposizione ottimale di capitale per massimizzare il tasso di crescita logaritmico del portafoglio nel tempo. La formula classica in forma matriciale è definita come:

$$\mathbf{f}^* = \Sigma^{-1} \boldsymbol{\mu} \quad (6.2)$$

- $\mathbf{f}^*$ è il vettore dei pesi ideali delle posizioni;
- Σ^{-1} è l'inversa della matrice di covarianza dei rendimenti;

- $\boldsymbol{\mu}$ è il vettore dei rendimenti attesi.

Questa formula è molto sensibile agli errori di stima degli input e tende a suggerire una leva finanziaria eccessivamente aggressiva e instabile che può facilmente portare alla distruzione del capitale in caso di errore. Per questo motivo sono state applicate le seguenti modifiche:

1. **Regolarizzazione della Matrice:** Se due o più titoli presentano un comportamento fortemente correlato o se il numero di campioni storici è ridotto, la matrice di covarianza diventa singolare (o quasi singolare). L'inversione di una tale matrice in JavaScript comporta gravi errori di precisione numerica o il blocco dell'applicazione. Per garantire la stabilità computazionale, viene applicata una regolarizzazione sui parametri fuori dalla diagonale principale:

$$\Sigma_{\text{shrunk}}(i, j) = \begin{cases} \Sigma(i, j) & \text{se } i = j \quad (\text{diagonale principale: varianze inalterate}) \\ (1 - \delta) \cdot \Sigma(i, j) & \text{se } i \neq j \quad (\text{fuori diagonale: covarianze ridotte del 15\%}) \end{cases} \quad (6.3)$$

In questo modo, i titoli appaiono leggermente meno correlati tra loro, garantendo un'inversione matriciale numericamente stabile.

2. **Scalamento Half-Kelly:** Dopo aver calcolato il vettore $\mathbf{f}^*$, si applica uno scalamento dimezzandone i valori:

$$\mathbf{f}_{\text{half}} = \frac{1}{2} \mathbf{f}^* \quad (6.4)$$

Lo scopo è diminuire i drawdown estremi causati da una leva eccessiva. Dimezzare le posizioni riduce la varianza complessiva del portafoglio del 75%, sacrificando solo una quota minima del tasso di crescita teorico.

3. **Vincolo Long-Only:** La formula originale restituisce valori sia positivi (acquisto) sia negativi (vendita allo scoperto). Poiché lo short selling richiede garanzie sui margini e autorizzazioni complesse, si eliminano le posizioni corte azzerando le componenti negative e ottenendo pesi non negativi f_i^{long} .
4. **Cap della Leva Massima:** Il criterio di Kelly tende ad adottare una strategia di leva molto aggressiva. Si è stabilito quindi un limite massimo alla leva finanziaria $L_{\text{max}} = 2$. Se l'esposizione lorda complessiva $L = \sum f_i^{\text{long}}$ supera tale soglia, i pesi vengono riallocati proporzionalmente:

$$\mathbf{f}_{\text{finale}} = \begin{cases} \mathbf{f}^{\text{long}} & \text{se } L \leq L_{\text{max}} \\ \mathbf{f}^{\text{long}} \cdot \frac{L_{\text{max}}}{L} & \text{se } L > L_{\text{max}} \end{cases} \quad (6.5)$$

6.4.4 Volatility Targeting

Il concetto teorico di questa strategia è spiegato nei fondamenti teorici (Cap. 2). In questa sezione si discute la sua implementazione nell'app. L'implementazione segue tre passaggi principali:

1. **Volatilità del portafoglio (giornaliera e annualizzata):** Si calcola prima la deviazione standard giornaliera mediante la forma quadratica della matrice di covarianza Σ e del vettore dei pesi base $\mathbf{w}_{\text{base}}$, per poi annualizzarla moltiplicando per la radice dei giorni lavorativi ($\sqrt{252}$):

$$\sigma_{\text{port, giornaliera}} = \sqrt{\mathbf{w}_{\text{base}}^T \Sigma \mathbf{w}_{\text{base}}} \quad (6.6)$$

$$\sigma_{\text{port, annua}} = \sigma_{\text{port, giornaliera}} \times \sqrt{252} \quad (6.7)$$

2. **Fattore di scalamento:** Viene calcolato il coefficiente di riduzione scale:

$$\text{scale} = \min \left(1, \frac{\sigma_{\text{target}}}{\sigma_{\text{port, annua}}} \right) \quad (6.8)$$

L'operatore $\min(1, \dots)$ impedisce l'impiego della leva, evitando che l'esposizione superi il 100% del capitale nei periodi di bassa volatilità.

3. **Pesi scalati:** Ogni peso dell'allocazione base viene ridimensionato applicando il fattore di scalamento:

$$\mathbf{w}_{\text{finale}} = \mathbf{w}_{\text{base}} \times \text{scale} \quad (6.9)$$

La funzione gestisce anche il caso limite in cui la volatilità annualizzata sia nulla ($\sigma_{\text{port, annua}} = 0$), impostando il fattore di scala a 1 per evitare divisioni per zero.

Chapter 7

Conclusioni

7.1 Discussione: un risultato atteso nella letteratura

I risultati ottenuti da questi esperimenti non sono esiti isolati. Il modello non riesce ad ottenere dalle feature segnali abbastanza forti da riuscire a sbloccare una sorta di previsione o intelligenza che gli permetta di battere la formula su cui si basa. Questo accade perché l'unico segnale che il modello riesce a sfruttare è quello della volatilità, ed proprio il dato principale su cui si basa la formula statica di partenza. La volatilità infatti può essere predetta: essa infatti tende ad addensarsi e susseguirsi, fasi ad alta volatilità sono seguite da altre fasi simili e ugualmente succede per le fasi di bassa volatilità. Questa proprietà, nota come volatility clustering, è ben documentata [6] e rende la volatilità un segnale stimabile in anticipo. E proprio questa caratteristica che ha portato al basare il modello sul volatility timing utilizzando la formula del volatility targeting di Moreira Muir. Il segnale mancante che la formula non ha e che permetterebbe al modello di battere la formula di partenza è la predizione della direzione del mercato. Questo segnale però è già stato dimostrato non essere ottenibile, la prevedibilità dei rendimenti azionari è notoriamente debole e instabile. Nel paper Goyal et al. [8] infatti si dimostra che fuori-campione la quasi totalità delle strategie e delle variabili proposte dalla letteratura accademica fallisce nel battere una previsione banale basata sulla semplice media storica. In sostanza non ci sono ancora modi per riuscire a prevedere la direzione futura del mercato. L'impossibilità di prevedere la direzione del mercato specialmente nel DRL è data dal fatto che la maggior parte delle informazioni che si possono ottenere dagli scambi giornalieri è quasi interamente rumore. Quando i modelli cercano di estrarre delle informazioni rilevanti da questi dati finiscono solo per fare over-fitting sul backtest in cui è stato addestrato.

Tirando le conclusioni il fatto che il modello converga sulla formula di partenza non è del tutto un fallimento. Può essere visto come un esito informativo, esso sebbene confermi che il valore della strategia risieda nella sua struttura anziché nella parte appresa, dimostra che

l'agente in assenza di segnale sfruttabile ha imparato la scelta corretta, ovvero riprodurre la regola ottimale evitando di inseguire pattern inconcludenti e degenerare.

Infine va riconosciuto il confine di questa conclusione. Gu et al. [9] documentano che il machine learning aggiunge valore nell'asset pricing empirico, ma nella dimensione cross-sectional, ovvero la selezione tra molti titoli e attraverso segnali come momentum, liquidità e volatilità. E' esattamente la leva che qui è stata congelata: il valore dell'apprendimento, se esiste in questo dominio, va cercato nella selezione cross-asset e in universi meno efficienti, non in un overlay più sofisticato sulla sola esposizione.

Limiti. Le conclusioni valgono entro i confini descritti nell'introduzione (Sez. 1.2.1): universo large-cap, selezione congelata, obiettivo di contenimento del drawdown e periodi considerati.

Contributo di questo lavoro. A differenza della maggior parte dei lavori, che riportano un singolo backtest vincente ma fragile fuori campione [1], in questo lavoro sono state testate diverse configurazioni su vari panieri e periodi, isolando le parti apprendibili del modello per capirne valore e limiti. Le accortezze che sono state fatte sono:

- L'apprendimento non sostituisce la strategia del volatility targeting e questo permette un confronto pulito tra ciò che la rete impara e quello che la formula fornisce.
- Si è scelto di valutare ogni componente apprendibile isolandolo e confrontandolo con la sua controparte strutturale.
- Le metriche sono riportate come mediana multi-seed, non si è selezionato l'esito più favorevole.
- Verifica di generalità in cui il modello è stato testato su panieri mai visti in addestramento.

Infine tutti quelle che sembravano azioni vantaggiose del modello sono state analizzate per capirne le cause, e se fossero o meno date da un effettiva intelligenza del modello o semplicemente dalla sua maggiore conservatività.

Per concludere il contributo non è dimostrare che l'apprendimento batte una regola, ma **quantificare con rigore, tramite ablation e controlli, il limite dell'adattività appresa** in un dominio a basso rapporto segnale/rumore e spiegarne la ragione economica.

Bibliography

- [1] David H. Bailey, Jonathan M. Borwein, Marcos López de Prado, and Qiji Jim Zhu. Pseudo-mathematics and financial charlatanism: The effects of backtest overfitting on out-of-sample performance. *Notices of the American Mathematical Society*, 61(5): 458–471, 2014.
- [2] Carlos Betancourt and Wen-Hui Chen. Deep reinforcement learning for portfolio management of markets with a dynamic number of assets. *Expert Systems with Applications*, 164:114002, 2021. doi: 10.1016/j.eswa.2020.114002.
- [3] Gregory Blotnick. The power of position sizing in portfolio management. SSRN 5363482, 7 2025. Working paper.
- [4] Jiayi Chen, Jing Li, and Guiling Wang. Mars: A meta-adaptive reinforcement learning framework for risk-aware multi-agent portfolio management. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.
- [5] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.
- [6] Robert F. Engle. Autoregressive conditional heteroscedasticity with estimates of the variance of united kingdom inflation. *Econometrica*, 50(4):987–1007, 1982.
- [7] Jeff Fleming, Chris Kirby, and Barbara Ostdiek. The economic value of volatility timing. *The Journal of Finance*, 56(1):329–352, 2001.
- [8] Amit Goyal, Ivo Welch, and Athanasse Zafirov. A comprehensive 2022 look at the empirical performance of equity premium prediction. *The Review of Financial Studies*, 37(11):3490–3557, 2024.
- [9] Shihao Gu, Bryan Kelly, and Dacheng Xiu. Empirical asset pricing via machine learning. *The Review of Financial Studies*, 33(5):2223–2273, 2020.

- [10] Zhengyao Jiang, Dixing Xu, and Jinjun Liang. A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint arXiv:1706.10059*, 2017.
- [11] Nak Hyun Jung and Taeyeon Oh. Factor-based deep reinforcement learning for asset allocation: Comparative analysis of static and dynamic beta reward designs. *PLoS ONE*, 20(12):e0332779, 2025.
- [12] Emmanuel Lwele, Sabuni Emmanuel, and Sitali Gabriel Sitali. Risk-aware deep reinforcement learning for dynamic portfolio optimization. *arXiv preprint arXiv:2511.11481*, 2025.
- [13] Alan Moreira and Tyler Muir. Volatility-managed portfolios. *The Journal of Finance*, 72(4):1611–1644, 2017.
- [14] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.
- [15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [16] Zhicheng Wang, Biwei Huang, Shikui Tu, Kun Zhang, and Lei Xu. Deeptrader: A deep reinforcement learning approach for risk-return balanced portfolio management with market conditions embedding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 643–650, 2021.
- [17] Zihao Zhang, Stefan Zohren, and Stephen Roberts. Deep learning for portfolio optimization. *The Journal of Financial Data Science*, 2(4):8–20, 2020.

Acknowledgment

